B.Tech. BCSE497J - Project-I

AN AUTOMATED DEEP LEARNING FRAMEWORK FOR EARLY DETECTION OF SKIN CANCER

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCB0165 ALAN JOHN ANAND

23BCE2002 SUJAN M

23BCE2107 RAMACHANDRAN SIVASANKAR

Under the Supervision of

Dr. Krishna Rani Samal

Associate Professor

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled An Automated Deep Learning Framework for Early Detection of Skin Cancer submitted by us, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by us under the supervision of Dr. Krishna Rani Samal.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidates

CERTIFICATE

This is to certify that the project entitled An Automated Deep Learning Framework for Early Detection of Skin Cancer submitted by **ALAN JOHN ANAND (23BCB0165), SUJAN M (23BCE2002) and RAMACHANDRAN SIVASANKAR (23BCE2107)**, **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

Head of the Department

B.Tech. Computer Science and Engineering

ACKNOWLEDGEMENTS

We would like to thank the management of Vellore Institute of Technology (VIT), for giving us the chance to do this project, and for the facilities of the institute which we used all through the semester.

We thank Dr. Jaisankar N, Dean of the School of Computer Science and Engineering (SCOPE), for his support and the encouragement he gives to the student projects in our school.

We are thankful to the Head of the Department of Computer Science and Engineering, for the support from department side and the guidance given to us in the review stages of the project.

We are very grateful to our project guide, Dr. Krishna Rani Samal, for her regular guidance and feedback. In the reviews she asked us questions which made us to look again at our evaluation, mainly how we handled the class imbalance and how we reported the missed cancers, and because of that this report became better.

We also thank our friends and families for supporting us during the project.

Alan John Anand (23BCB0165)
Sujan M (23BCE2002)
Ramachandran Sivasankar (23BCE2107)

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**x**
1.	**INTRODUCTION**	**1**
	1.1 Background	**1**
	1.2 Motivations	**1**
	1.3 Scope of the Project	**2**
2.	**PROJECT DESCRIPTION AND GOALS**	**3**
	2.1 Literature Review	**3**
	2.1.1 From hand-crafted features to deep learning	**3**
	2.1.2 Transfer learning and DenseNet	**3**
	2.1.3 Datasets and data leakage	**3**
	2.1.4 Class imbalance and evaluation	**3**
	2.2 Research Gap	**4**
	2.3 Objectives	**5**
	2.4 Problem Statement	**5**
	2.5 Project Plan	**5**
3.	**TECHNICAL SPECIFICATION**	**7**
	3.1 Requirements	**7**
	3.1.1 Functional	**7**
	3.1.2 Non-Functional	**7**
	3.2 Feasibility Study	**7**
	3.2.1 Technical Feasibility	**7**
	3.2.2 Economic Feasibility	**8**
	3.2.3 Social Feasibility	**8**
	3.3 System Specification	**8**
	3.3.1 Hardware Specification	**8**
	3.3.2 Software Specification	**8**
4.	**DESIGN APPROACH AND DETAILS**	**9**
	4.1 System Architecture	**9**
	4.2 Design	**11**
	4.2.1 Data Flow Diagram	**11**
	4.2.2 Use Case Diagram	**12**
	4.2.3 Class Diagram	**13**

4.2.4 Sequence Diagram 14
5. METHODOLOGY AND TESTING 16
5.1 Module Description **16**
5.1.1 Dataset and patient-level split **16**
5.1.2 Preprocessing and augmentation **17**
5.1.3 Model **17**
5.1.4 Experiment 1: baseline with weighted BCE **18**
5.1.5 Experiment 2: fine-tuning with weighted BCE **18**
5.1.6 Experiment 3: WeightedRandomSampler **18**
5.1.7 Experiment 4: focal loss **19**
5.1.8 Experiment 5: random oversampling **19**
5.1.9 Threshold selection **19**
5.1.10 Evaluation metrics **19**
5.2 Testing **20**
6. PROJECT DEMONSTRATION 22
6.1 Running an Experiment **22**
6.2 Threshold Selection and Final Test Run **22**
6.3 Output Files **22**
7. RESULT AND DISCUSSION (COST ANALYSIS) 24
7.1 Training and Validation Behaviour **24**
7.2 Test Results **25**
7.3 Effect of the Decision Threshold **28**
7.4 Accuracy and Uncertainty **29**
7.5 Comparison of the Strategies **29**
7.6 Discussion **30**
7.7 Cost Analysis **30**
8. CONCLUSION 32
9. REFERENCES 33
APPENDIX A – SAMPLE CODE 36

List of Figures

Figure No.	Title	Page No.
2.1	Project plan (semester weeks)	5
4.1	System architecture of DermInsight AI (implemented parts and proposed parts)	10
4.2	Starting point and main settings of the five experiments	11
4.3	Data flow diagram, level 0 (context)	12
4.4	Data flow diagram, level 1	12
4.5	Use case diagram (implemented and proposed)	13
4.6	Class diagram of the implemented code	14
4.7	Sequence diagram of training and evaluation (Exp 2)	15
5.1	Class distribution of the ISIC 2020 training data (logarithmic scale)	16
5.2	Benign and malignant images in each split	17
5.3	Evaluation protocol followed in every experiment	20
6.1	Predicted probabilities on the test set, Exp 3 (saved output)	23
7.1	Validation ROC-AUC by epoch (circles mark the saved checkpoint)	24
7.2	Training and validation loss of Exp 1 (x-axis: epoch index 0 to 2)	25
7.3	Test confusion matrices of the five experiments (original saved plots)	26
7.4	Test ROC curves (Exp 3–5) and operating points (Exp 1–2)	27
7.5	Test precision-recall curves (Exp 3–5) and operating points (Exp 1–2)	27
7.6	Sensitivity, specificity, precision and F1 of each experiment on the test set	28
7.7	Sensitivity and specificity against the threshold on the test set (descriptive only)	28
7.8	Predicted probabilities on the test set, Exp 4 (focal loss)	29

List of Tables

Table No.	Title	Page No.
2.1	Summary of the most relevant prior work	4
2.2	Project phases and deliverables	6
3.1	Functional requirements	7
3.2	Non-functional requirements	7
3.3	Hardware used	8
3.4	Software used	8
4.1	Pipeline parts and their files	10
5.1	Patient-level split of the data	16
5.2	Preprocessing and augmentation settings	17
5.3	Training settings of the five experiments	20
5.4	Checks done on the pipeline and the results	20
6.1	Output files of each experiment	23
7.1	Validation results of the five experiments	24
7.2	Results on the test set (N = 4,920; 68 malignant)	25
7.3	Exp 5 on the test set at two thresholds	29
7.4	Test sensitivity with 95% Wilson intervals (68 malignant images)	29
7.5	Cost of the project	31
7.6	Illustrative error cost on the test set (assumed unit costs, not real prices)	31

List of Abbreviations

AdamW	Adam optimiser with decoupled weight decay
AP	Average Precision
AUC	Area Under the Curve
BCE	Binary Cross-Entropy
CI	Confidence Interval
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DFD	Data Flow Diagram
Exp	Experiment
F1	Harmonic mean of precision and recall
FN	False Negative
FP	False Positive
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
ISIC	International Skin Imaging Collaboration
JSON	JavaScript Object Notation
PR	Precision-Recall
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
SIIM	Society for Imaging Informatics in Medicine
TN	True Negative
TP	True Positive
UML	Unified Modeling Language
VIT	Vellore Institute of Technology
W-BCE	Weighted Binary Cross-Entropy
WRS	WeightedRandomSampler (PyTorch)

Symbols and Notations

x	Input image tensor of size $3 \times 224 \times 224$
y	True label, 0 = benign, 1 = malignant
z	Raw output (logit) of the linear head
$\hat{p}$	Predicted probability of malignancy, $\sigma(z)$
$\sigma(\cdot)$	Logistic sigmoid function
τ	Decision threshold on $\hat{p}$
$w+$	Weight on the positive class in W-BCE (50.2067)
N_{neg}, N_{pos}	Benign and malignant counts in the training split
p_t	Predicted probability of the true class (focal loss)
α	Class balance factor in focal loss (0.75)
γ	Focusing exponent in focal loss (2.0)
η	Learning rate (1×10^{-4})
λ	Weight decay in AdamW (1×10^{-4})
μ, s	ImageNet channel mean and standard deviation used for normalisation

ABSTRACT

Melanoma is only around one percent of the skin cancers, but most of the deaths from skin cancer is because of it [2]. If it is found when it is still localised, the five-year relative survival is above 99 percent, and once it spread to distant organs, it comes down to about 36 percent [3]. So finding it early is very important. Deep learning models can give a malignancy score for a dermoscopic image, but the real screening data is very much imbalanced. We counted only 584 malignant images out of the 33,126 in SIIM-ISIC 2020, which is 1.76%. So if a model just says benign for every image, it still gets 98.24% accuracy and catches no cancer at all.

For this we built DermInsight AI. It takes only the dermoscopic image, no patient details, and says benign or malignant. We split the data by patient. Training got 22,787 images from 1,439 patients, validation got 5,419 images from 307 patients and test got 4,920 images from 310 patients, and we checked in code that no patient comes in two splits. All five of our experiments use the same ImageNet-pretrained DenseNet201 with one linear output on top. Exp 1 trains only the head with weighted binary cross-entropy, Exp 2 fine-tunes the full network with the same loss, Exp 3 uses a WeightedRandomSampler, Exp 4 uses focal loss and Exp 5 does random oversampling of the training split. We picked the best epoch of each run by its validation ROC-AUC. Thresholds also we tuned on validation only, never on test.

On the same 4,920 test images, the test ROC-AUC came between 0.800 and 0.871, and random oversampling gave the best value. At a fixed threshold the results was much weaker. The baseline caught 62 of the 68 cancers, but it also flagged 2,819 benign images (specificity 41.90%). The runs which had specificity above 98% found only 10 to 15 of the 68 cancers. WeightedRandomSampler run gave the best balance, with 36.76% sensitivity at 96.85% specificity. There are only 68 malignant test images, so these numbers are uncertain, and this work is a research prototype and it is not a clinical tool. Metadata fusion, Grad-CAM, probability calibration and testing on outside data is planned for Project-II.

Keywords: *melanoma, dermoscopy, deep learning, DenseNet201, class imbalance, ISIC 2020, patient-level split, threshold selection.*

CHAPTER 1

1. INTRODUCTION

1.1 Background

Skin cancer is one of the most common cancer, and among them melanoma is the most dangerous type. As per the American Cancer Society, melanoma is only about 1% of skin cancers, but a large majority of skin cancer deaths come from it [2]. The stage when it is diagnosed matters a lot. For people diagnosed between 2015 and 2021, the five-year relative survival was more than 99% when the melanoma was localised, 76% when it spread to nearby areas, and 36% when it reached distant organs [3]. If a suspicious lesion is caught while it is still thin and local, many times a simple excision is enough.

Doctors normally see a lesion using a dermatoscope. With its magnification and polarised or immersion light, the doctor can see things under the skin surface, like the pigment network and the vessel patterns. To read these images properly, a doctor needs training and practice. An early melanoma can look almost same as a normal mole, and many patients will first show their lesion to a general practitioner and not to a dermatologist. Some reader studies showed that convolutional neural networks (CNNs) trained on dermoscopic images can come close to the level of dermatologists, on test sets of images [13, 16, 4]. This is the main reason why automatic image analysis is studied as a support tool for screening.

Most of the published works report good numbers on datasets which are fairly balanced or curated. Real screening data is not like that. In the SIIM-ISIC 2020 challenge data, which is collected from routine skin checks of 2,056 patients, only 584 images out of 33,126 are malignant [26]. When we work with this kind of dataset there are two practical problems, and this project focus on them. One is that the model has to learn from very few positive examples, and second is that the evaluation has to be honest about how many cancers the model actually finds.

1.2 Motivations

- **Very few positive examples.** With 1.76% malignant images, a classifier which always says benign is 98.24% accurate and it finds no cancer at all. Normal training on this kind of data pushes the model towards majority class, so the imbalance has to be handled on purpose.
- **Data leakage between splits.** Many patients in ISIC 2020 have more than one image. If the images are split randomly, the same person's skin, hair, lighting and camera can come in both training and test data, and the test score will become too optimistic [6]. We wanted a split where this cannot happen.
- **Ranking is not the same as deciding.** A model can rank the malignant images above benign ones quite well (good ROC-AUC), and still it can miss most of the cancers at the threshold which is finally used. So we wanted to measure both of them and report the exact number of cancers missed.
- **Limited compute.** We trained on free Google Colab sessions and a normal laptop, and Exp 1 even ran on a CPU. So we started from a pretrained DenseNet201 and kept most runs to 3 to 5 epochs, enough to compare the five methods.

1.3 Scope of the Project

This project is a research pipeline for binary classification of dermoscopic images, that is benign or malignant. The scope of Project-I is given below:

- We audit the SIIM-ISIC 2020 training metadata and divide all the 33,126 images into training, validation and test sets patient wise, and a code check makes sure that no patient is shared between sets.
- We build a PyTorch input pipeline, which does resizing, ImageNet normalisation and light augmentation only for training images.
- We use an ImageNet-pretrained DenseNet201 with a single linear output unit as the classifier.
- Five experiments, each with a different way of handling the ~55:1 imbalance. Exp 1 and Exp 2 use weighted loss (frozen backbone, then full fine-tuning), Exp 3 a WeightedRandomSampler, Exp 4 focal loss and Exp 5 random oversampling.
- We pick the best epoch by validation ROC-AUC. Tuned thresholds (0.86 in Exp 2, 0.7296 in Exp 5) also come from validation. Then each model is run one time on the same 4,920 test images.
- Every run leaves behind its checkpoint, prediction files, metric files and plots, and we checked each number in this report against those files.

Some things are out of scope for Project-I. These are using patient metadata (age, sex, body site) inside the model, Grad-CAM explanations, probability calibration, testing on data from other hospitals, and any application for users. In the design diagrams of Chapter 4 we marked them as proposed parts, and we plan to build them in Project-II.

CHAPTER 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

We reviewed the previous work in four areas, which directly affected our design. They are the shift from hand-crafted features to CNNs, transfer learning and DenseNet architecture, datasets used for skin lesion classification, and the methods for handling and evaluating class imbalance. Table 2.1 gives a summary of the studies which we used the most.

2.1.1 From hand-crafted features to deep learning

The early computer-aided systems was copying the rules that dermatologists follow, like the ABCD rule and the seven-point checklist. They segment the lesion first, then measure shape, colour and texture features and give them to a classifier like SVM [7, 1]. These systems were easy to explain, but they depended a lot on correct segmentation and on the imaging setup. Esteva et al. [13] trained an Inception-v3 network starting from ImageNet weights on about 129,000 clinical images, and it performed on par with board-certified dermatologists on biopsy-proven test images. Haenssle et al. [16] and Brinker et al. [4] also reported similar results in reader studies with dermoscopic images, and Tschandl et al. [30] found that in an open web-based study the best algorithms did better than most of the human readers on same images. All these studies used fixed image sets, so it shows what a model can do on that data, and not how it will behave in a clinic.

2.1.2 Transfer learning and DenseNet

Medical image datasets are small when compared with ImageNet [12], and malignant cases are even less. Yosinski et al. [32] showed that the features learned in early layers of a CNN will transfer well between tasks, while the later layers are more specific to the task. That is why nearly all skin lesion works start from ImageNet weights and then fine-tune. We selected DenseNet201 [18]. In a dense block, each layer gets the feature maps of all the earlier layers, so low-level features like edges and colour changes are still available to the deeper layers and they get reused. It has less parameters than a ResNet [17] of similar depth, and this matters when training time is limited.

2.1.3 Datasets and data leakage

The main public dermoscopy benchmarks come from the ISIC archive [9, 29]. HAM10000 [29] has seven diagnosis classes and the share of melanoma in it is fairly high. The SIIM-ISIC 2020 dataset [26] is different, because it was collected from screening of patients having many lesions, and every image has a patient identifier. So it is closer to a real screening population, where the share of melanoma is very low. Cassidy et al. [6] analysed the ISIC datasets and found many duplicate images within and between datasets, including duplicates between training and test sets. Because of their work we split the data by patient, and also we list the duplicate images as one remaining limitation.

2.1.4 Class imbalance and evaluation

Buda et al. [5] studied class imbalance in CNNs, and they found that oversampling the minority class was the best method in most of their settings and it did not cause overfitting of the CNNs. Johnson and Khoshgoftaar [19] grouped these methods into data-level methods (resampling) and algorithm-level methods (changing the loss). From the first group we used random oversampling

and a weighted sampler. SMOTE [8] makes synthetic samples, but it was designed for feature vectors, so for images it is less natural. From the second group we used a positive class weight in binary cross-entropy, and focal loss [21], which reduces the loss of easy examples so that the training will focus on hard ones. For evaluation, Saito and Rehmsmeier [27] showed that precision-recall plots give more information than ROC plots when the positives are rare, and Davis and Goadrich [11] explained how these two curves are related. Guo et al. [15] showed that modern deep networks are many times poorly calibrated, so a predicted probability should not be read as a risk directly.

Table 2.1: Summary of the most relevant prior work

Ref.	Work	What it shows	Use in this project
[13]	Esteva et al., 2017	CNN started from ImageNet weights, it matched dermatologists on a fixed test set	Supports transfer learning from ImageNet
[16], [4]	Haenssle 2018; Brinker 2019	CNNs did well when compared with dermatologists on dermoscopic images	Motivation for image-based screening support
[18]	Huang et al., 2017	DenseNet reuses the features by dense connections	Backbone of all five experiments
[26]	Rotemberg et al., 2021	ISIC 2020 dataset with patient IDs, 1.76% melanoma	Dataset and patient-level split
[6]	Cassidy et al., 2022	ISIC data has duplicates and train/test overlap	Reason for patient-level split; duplicate limitation
[5], [19]	Buda 2018; Johnson 2019	Oversampling and loss weighting to handle imbalance	Exp 1, 2, 3 and 5 design
[21]	Lin et al., 2017	Focal loss gives less weight to easy examples	Exp 4 design
[27], [11]	Saito 2015; Davis 2006	PR analysis tells more when positives are rare	Report AP and PR curves, not only ROC-AUC
[15]	Guo et al., 2017	Deep networks are many times miscalibrated	Do not read $\hat{p}$ as a risk; future work

2.2 Research Gap

Each part which we used is already known from earlier work. The gap is in how these parts are evaluated together on realistic data. From the literature and from our early runs, we noted these points:

1. Many studies report accuracy or ROC-AUC at the default threshold of 0.5 on balanced or curated data, and they do not show how many cancers are missed when malignant cases are below 2% of data.
2. Patient-level separation of training and test data is often not mentioned, even though ISIC data has many images per patient and also duplicate images [6].
3. Imbalance methods are mostly reported one at a time, on different splits and with different evaluation rules, so comparing them is difficult.
4. How the decision threshold was chosen is rarely described, and sometimes it is not clear if it was chosen on validation data or test data.

We are not proposing a new network in this project. What we do is compare five imbalance strategies on one patient-level split. Backbone, test set and reporting stay the same for all of them. For every model we give the full confusion matrix, because the missed cancer count (43 of 68 in

Exp 3, for example) is the first number a reader should check.

2.3 Objectives

- To make a patient-level train, validation and test split of the SIIM-ISIC 2020 data, and check in code that no patient is shared between the splits.
- To build a binary classifier based on DenseNet201, along with a training and evaluation pipeline in PyTorch which can be reproduced.
- To compare loss weighting, weighted sampling, focal loss and random oversampling with the same split and the same test protocol.
- To pick checkpoints and thresholds from the 5,419 validation images only, and touch the 4,920 test images just once per model.
- To report ROC-AUC and average precision for each experiment, along with sensitivity, specificity, precision, F1 and the confusion matrix, all cross-checked with the saved metric files.
- To list what did not work or could not be confirmed, like the missing scripts of Exp 3, 4 and 5, and use that list to plan Project-II.

2.4 Problem Statement

Given a dermoscopic image x of a skin lesion, our model has to output a probability $\hat{p} = \sigma(f(x))$ that the lesion is malignant. The decision is then $\hat{y} = 1$ if $\hat{p} \geq \tau$, otherwise 0. We learn f on the training split, and τ is fixed before we look at the test set at all. Two things make this hard for us. Our training split has 22,342 benign images against only 445 malignant ones, about 50 to 1. Also, all the images of one patient have to stay inside one split, so the test score is on patients the model never saw. The aim is to find which imbalance strategy will give the most useful balance between cancers found and false alarms on unseen patients, and to say honestly how far these results are from clinical use.

2.5 Project Plan

We planned the work in four phases over the semester (Fig. 2.1). Every phase ended with some files, which was needed for the next phase, and they are listed in Table 2.2.

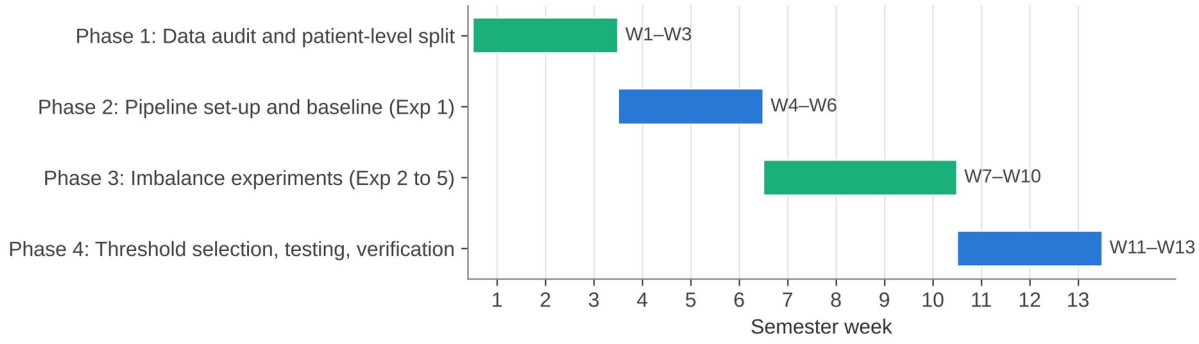


Figure 2.1: Project plan (semester weeks)

Table 2.2: Project phases and deliverables

Phase	Weeks	Work	Deliverable
1	1–3	Download metadata, audit labels and patient IDs, make patient-level split	train.csv, val.csv, test.csv
2	4–6	Dataset class, transforms, DenseNet201 model, baseline training (Exp 1)	densenet201_baseline.py, Exp 1 checkpoint and metrics
3	7–10	Weighted BCE fine-tuning, sampler, focal loss and oversampling runs (Exp 2–5)	Checkpoints, histories and predictions for Exp 2–5
4	11–13	Threshold selection on validation, final test runs, plots, checking of results, report	Metric JSON/CSV files, plots, this report

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

Table 3.1: Functional requirements

ID	Requirement	Where it is done
FR1	Read the split CSV files and load every image using its image_name, along with target label.	DermImageDataset, ISICDataset
FR2	Stop the run if any patient ID comes in more than one split.	assert checks in train_model()
FR3	Resize images to 224 × 224 and normalise them with ImageNet mean and standard deviation, and augment only the training images.	build_transform()
FR4	Give one logit for each image using a DenseNet201 backbone and a 1920 → 1 linear head.	DermInsightDenseNet201
FR5	Train using the selected imbalance strategy (loss weight, sampler, focal loss or oversampling).	Experiment scripts and settings
FR6	After every epoch evaluate on validation set and save the checkpoint which has best validation ROC-AUC.	Training loops of Exp 1 and Exp 2; checkpoints of Exp 3–5
FR7	When needed, select the decision threshold on validation set (best F1 over 0.01 to 0.99).	stage2b_final_evaluation.py; Exp 5 evaluation
FR8	On the test set, compute ROC-AUC, AP, accuracy, precision, sensitivity, specificity, F1 and the confusion matrix.	calculate_binary_metrics(), evaluation scripts
FR9	For each experiment, save metrics as JSON or CSV, predictions as CSV and plots as PNG files.	results/EXP_0x folders

3.1.2 Non-Functional

Table 3.2: Non-functional requirements

ID	Requirement	How it is met
NFR1	Reproducibility	Seed 42 for Python, NumPy and PyTorch; deterministic cuDNN in the baseline script; fixed split files
NFR2	No leakage	Patient-level split; validation-only threshold choice; test set used once per model
NFR3	Traceability	Each experiment has its own folder for code, checkpoint and results; organisation manifest stores SHA-256 hashes of copied files
NFR4	Low cost	Only open-source software and free cloud notebook runtimes
NFR5	Modularity	Dataset, model, metric and plotting code are separate functions and classes
NFR6	Honest reporting	Full confusion matrix and exact counts of missed cancers for every model

3.2 Feasibility Study

3.2.1 Technical Feasibility

All the parts are well supported by the existing libraries. PyTorch [24] and torchvision give DenseNet201 with ImageNet weights, the data loaders, the WeightedRandomSampler and the BCEWithLogitsLoss with a pos_weight option. scikit-learn [25] gives the functions for ROC-

AUC, average precision and confusion matrix. The input size of 224 × 224 and batch size of 32 or 64 fit on a free cloud GPU. As an example of training time, one epoch of Exp 3 over the full training split took around 610 to 620 seconds, as it is recorded in its training history file.

3.2.2 Economic Feasibility

We did not pay for any software or hardware. The dataset is public, the libraries are open source, and training was done on free Google Colab sessions, where the images were taken from a public Kaggle copy of the ISIC 2020 data. What it cost us was mainly our own time. Section 7.7 gives the cost details.

3.2.3 Social Feasibility

A screening aid can help the doctors in places where dermatologists are less, by pointing out lesions that need a closer look. At the same time, a wrong result can harm a patient, either by giving false reassurance or by an unnecessary biopsy. So DermInsight AI is only a research prototype now. Even Exp 3 misses 43 of the 68 test cancers, so it must not diagnose anyone. Real use would need clinical trials and approval first, and even then a dermatologist has to make the final call.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.3: Hardware used

Item	Detail	Evidence
Exp 1 training	CPU (device recorded as "cpu")	densenet201_baseline_metrics.json
Exp 2–5 training and evaluation	Google Colab runtime with GPU when available; exact GPU model not recorded	Colab paths in scripts and summaries
Local workstation	Windows 11 laptop, used for organising files and writing the report	Project inventory notes
Storage	Google Drive project folder; checkpoints about 70 MB each (Exp 5: 209 MB, includes optimiser state)	models/ folder

3.3.2 Software Specification

Table 3.4: Software used

Software	Version	Use
Python	3.12 (local); Colab default for training	Programming language
PyTorch, torchvision	2.x (not pinned)	Model, data loading, training, DenseNet201 weights
scikit-learn	not pinned	ROC-AUC, average precision, confusion matrix, F1
pandas, NumPy	not pinned	Reading CSV files and handling arrays
Pillow (PIL)	not pinned	Opening JPEG images
Matplotlib	not pinned	Loss curves, ROC, PR and confusion matrix plots
Google Colab, Google Drive	web service	GPU runtime and file storage

There is no requirements file in the project folder, so the library versions were not recorded. We noted this as a reproducibility limitation in Section 7.6.

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

Fig. 4.1 shows the architecture of DermInsight AI as it is implemented in Project-I. There are four layers in it. Data layer holds the ISIC 2020 images and the metadata CSV, and it converts the metadata into three patient-level split files. In the input pipeline, our DermImageDataset class reads the three split CSVs, loads each .jpg and then resizes and normalises it. Model layer is the DenseNet201 backbone with a linear head, which is trained with one of the loss or sampling options. Storage and evaluation layer saves the best checkpoint, scores the validation and test images, applies the threshold and writes the metrics and plots. The dashed box at bottom shows the parts which are planned for Project-II, and these are not built yet.

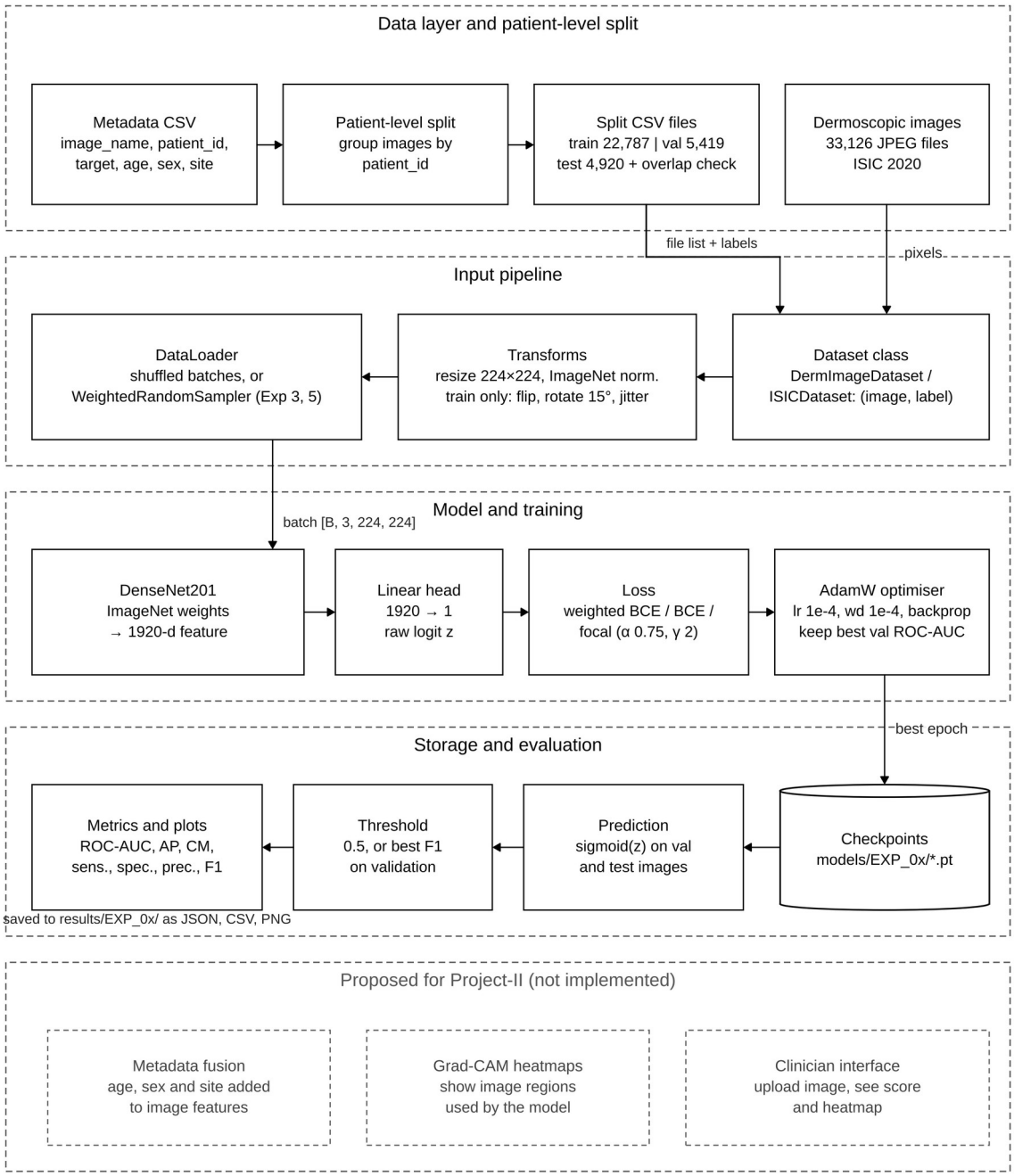


Figure 4.1: System architecture of DermInsight AI (implemented parts and proposed parts)

Table 4.1 connects each part of the architecture to the files in project folder. All five experiments share this structure, and they differ only in training choices, as shown in Fig. 4.2. Exp 2 is the only run which starts from another experiment, as it loads the Exp 1 checkpoint and trains all the layers. Other runs start from the ImageNet weights.

Table 4.1: Pipeline parts and their files

Part	**File or folder**	**Input**	**Output**
Patient split	data/processed/*.csv	ISIC_2020_Training_GroundTruth_v2.csv	train, val and test CSV files
Dataset and transforms	src/EXP_01_BASELINE/ densenet201_baseline.py	Split CSV, JPEG images	Tensors [B, 3, 224, 224] and labels
Model	DermInsightDenseNet201 class	Image tensor	Logit z per image

Part	File or folder	Input	Output
Training	Exp 1 and Exp 2 scripts	Batches, loss settings	Best checkpoint (.pt), history JSON
Threshold and test	stage2b_final_evaluation.py; Exp 5 evaluation files	Checkpoint, val and test sets	Threshold, test metrics JSON or CSV
Plots	plot functions, stage2b_visual_results.py	Predictions, histories	PNG plots

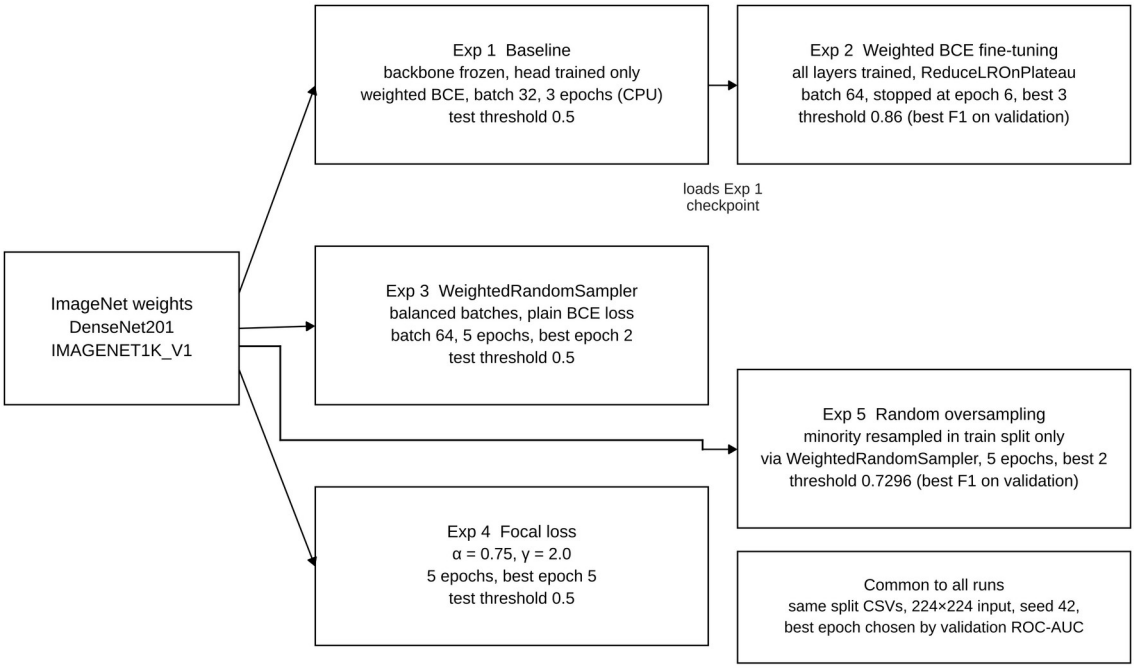


Figure 4.2: Starting point and main settings of the five experiments

4.2 Design

4.2.1 Data Flow Diagram

In the context diagram (Fig. 4.3) the whole pipeline is shown as one process. It takes images and metadata from the ISIC 2020 dataset and experiment settings from the researcher, and it gives back checkpoints, predictions, metrics and plots. The level 1 diagram (Fig. 4.4) breaks this process into six steps. Process 1.0 splits the metadata by patient and writes split files to store D3. Process 2.0 combines these split files with raw images from D1 and prepares tensor batches. Process 3.0 trains the model and writes best weights to D4. Process 4.0 uses these weights to score validation and test sets. Process 5.0 uses only validation probabilities for choosing the threshold, and process 6.0 applies it on the test probabilities and writes the results to D5.

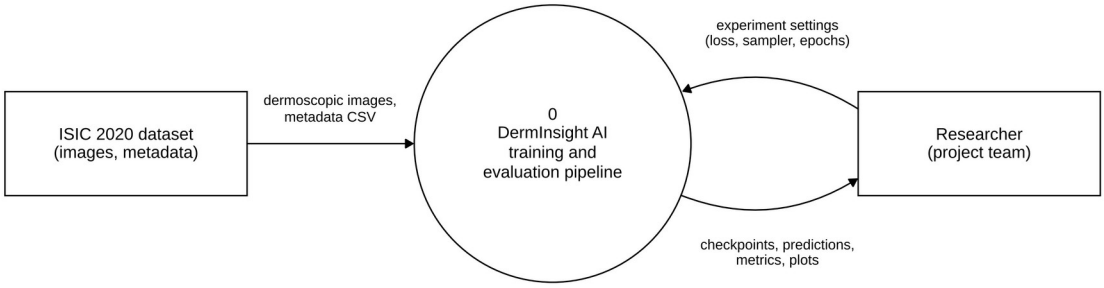


Figure 4.3: Data flow diagram, level 0 (context)

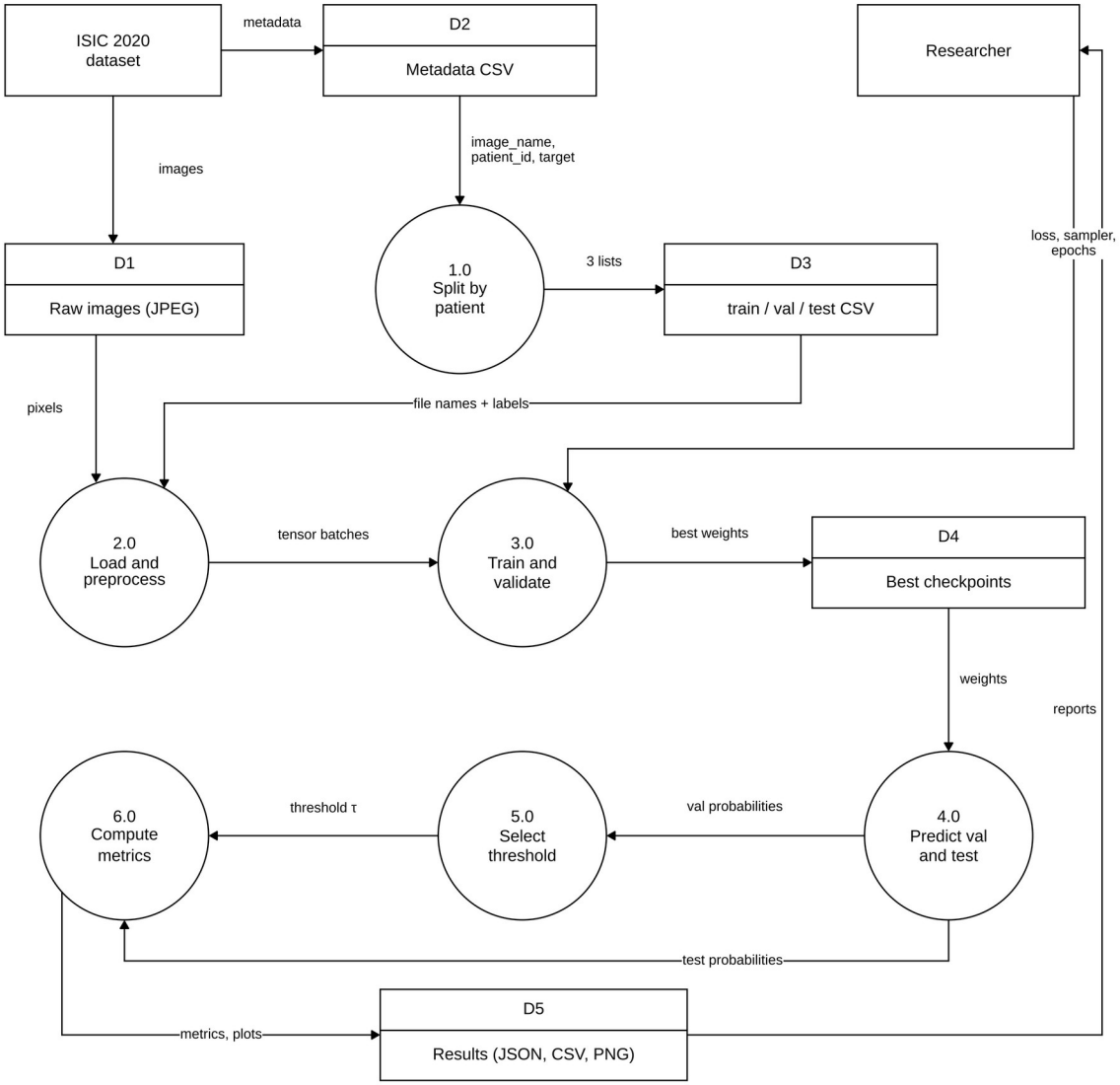


Figure 4.4: Data flow diagram, level 1

4.2.2 Use Case Diagram

In Project-I, the only real user is the researcher, that means a member of our team who runs the scripts (Fig. 4.5). The researcher prepares the splits, sets up an experiment, trains it, evaluates it and compares results. Training includes validation after every epoch and saving of best checkpoint.

Test evaluation always writes the metrics and plots. In Exp 2 and Exp 5 it also goes through an extra threshold selection step, drawn as an extend relation. We drew Google Colab as a second actor since validation and checkpoint saving actually happen there. Down at the bottom, a separate box holds use cases for a future clinician tool. We left it apart from the rest because nothing like that exists in Project-I yet.

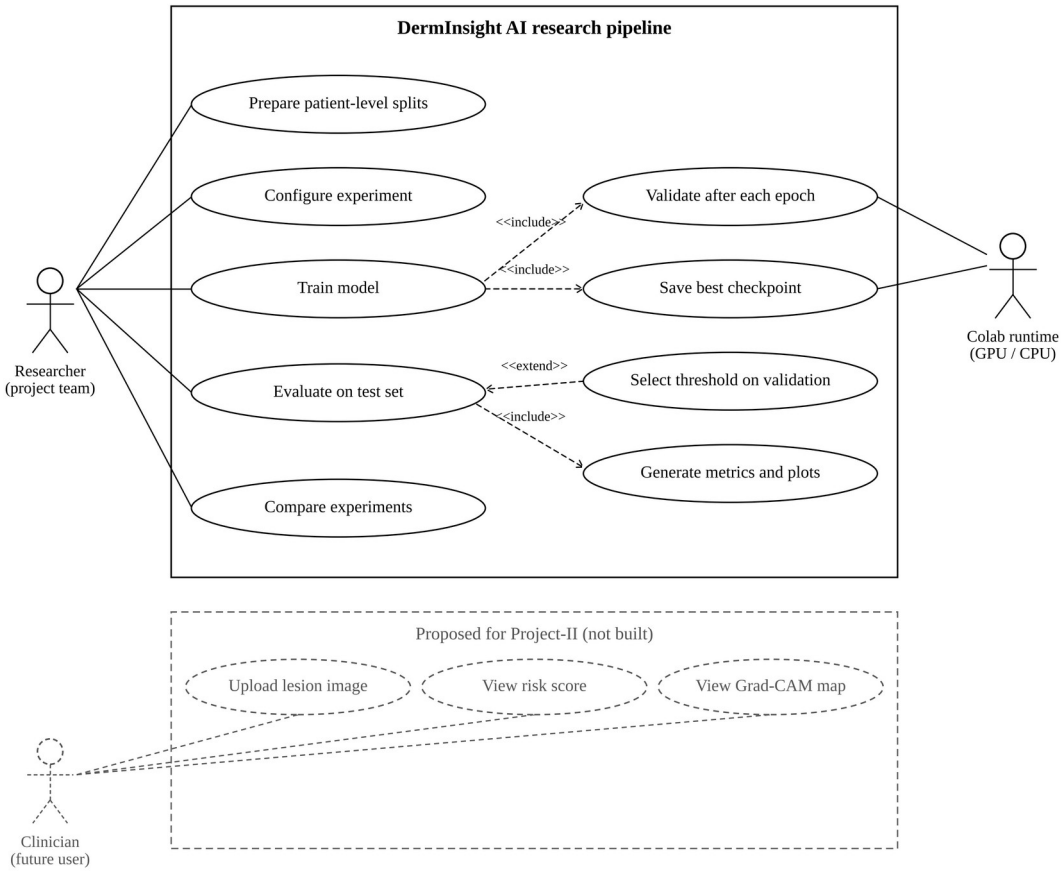


Figure 4.5: Use case diagram (implemented and proposed)

4.2.3 Class Diagram

Most of the code is script based, so Fig. 4.6 shows the real classes along with main scripts as «module» boxes. DermImageDataset and ISICDataset both extend the PyTorch Dataset class. The first one returns the image name also, and it helps when we check predictions. DermInsightDenseNet201 and DenseNet201Baseline both extend nn.Module and have the same two parts, which are a DenseNet201 backbone whose classifier is replaced by an identity layer, and a linear head with one output. The Exp 2 class is created with weights=None, because its weights come from the Exp 1 checkpoint. Training scripts of Exp 3, 4 and 5 are not in the project folder. We checked their saved checkpoints, and they have a DenseNet201 where the classifier is a single 1920 → 1 linear layer, so it is the same design.

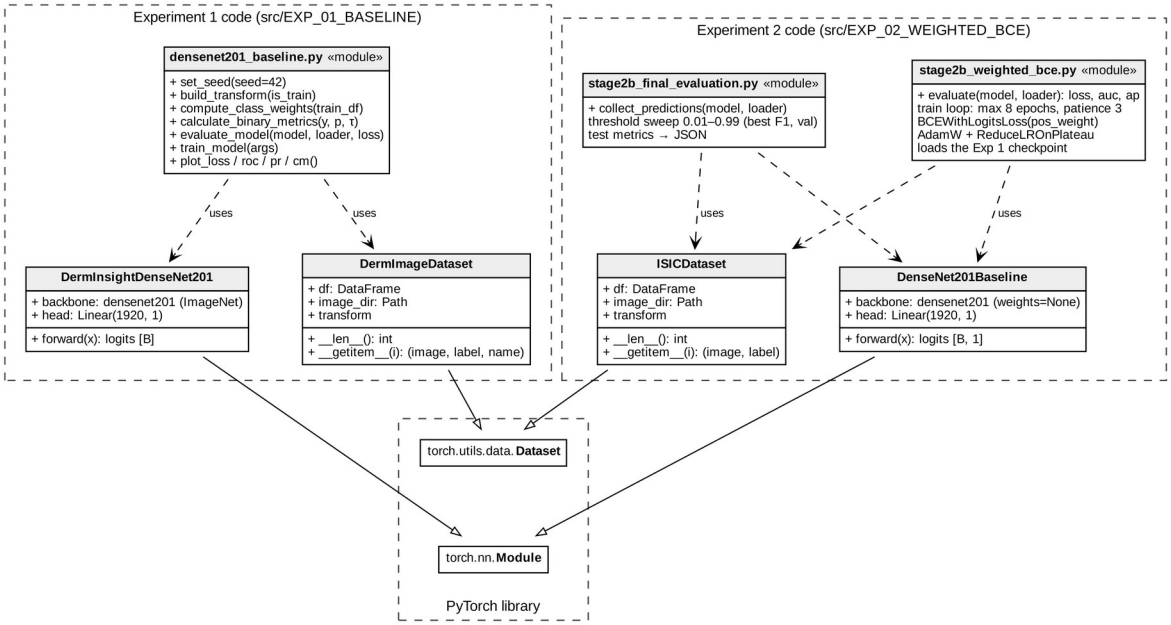


Figure 4.6: Class diagram of the implemented code

4.2.4 Sequence Diagram

Fig. 4.7 shows one full run of Exp 2, from training till the final test numbers. The training script loads Exp 1 weights first and then runs the epoch loop. Inside every epoch, the data loader gives batches to the model, weighted BCE loss is computed and AdamW updates the weights. At the end of every epoch we score the 5,419 validation images and pass the validation ROC-AUC to the learning rate scheduler. A new checkpoint is written only when this ROC-AUC beats the best value so far. The loop will stop after three epochs with no improvement. Then a separate evaluation script loads the best checkpoint, sweeps the threshold on validation set, applies the selected threshold (0.86) one time on test set and writes the result file.

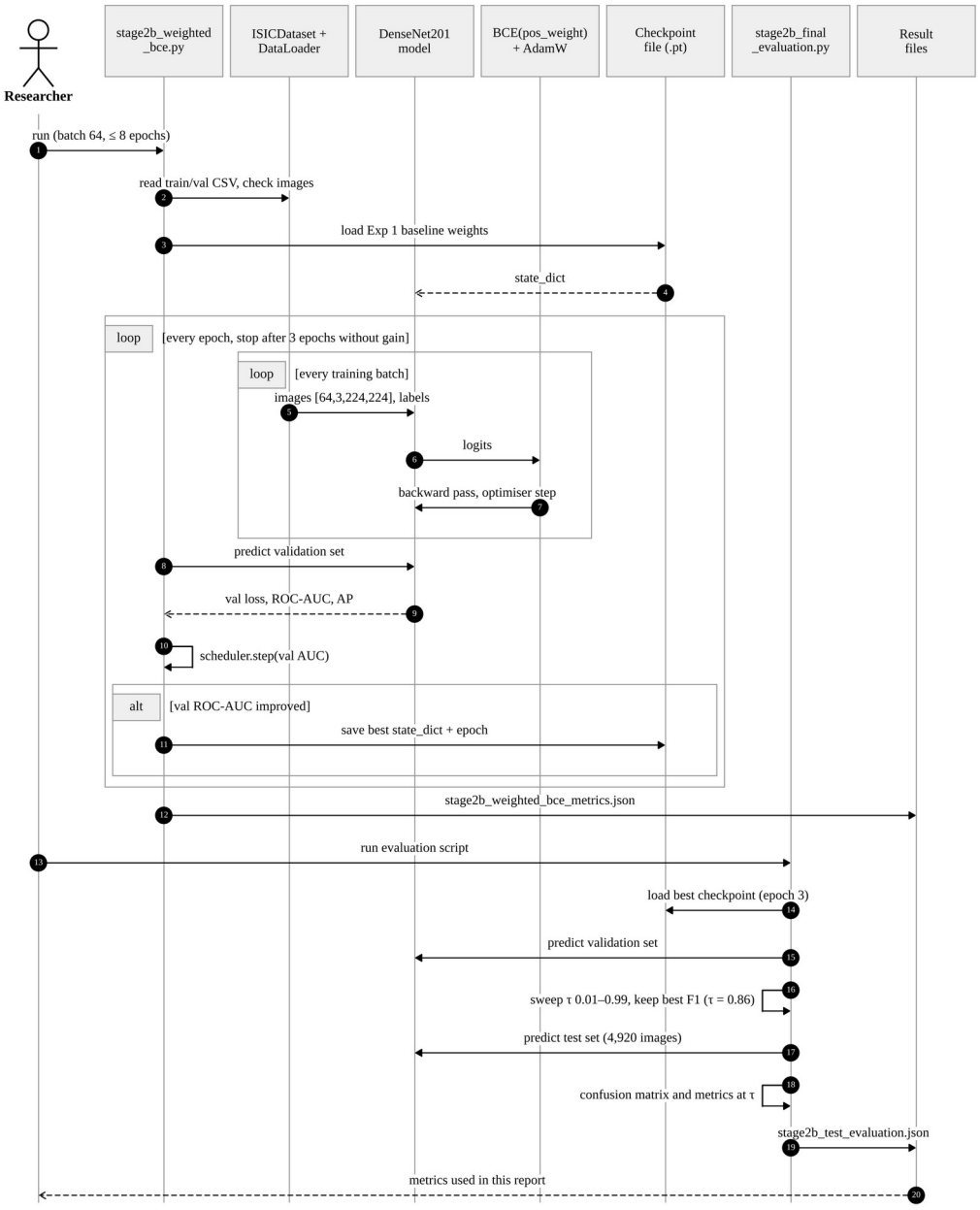


Figure 4.7: Sequence diagram of training and evaluation (Exp 2)

CHAPTER 5

5. METHODOLOGY AND TESTING

All the five experiments follow one protocol (Fig. 5.3). The model is trained on training split, the epoch which has best validation ROC-AUC is kept, and the decision threshold is either the default 0.5 or the value with best F1 score on validation set. The test set is scored only once, with the threshold fixed before. Below sections describe each module, and after that the checks we did for testing the pipeline and the reported numbers.

5.1 Module Description

5.1.1 Dataset and patient-level split

We used the training part of SIIM-ISIC 2020 challenge data [26], because the labels of official test set are not public. The ground-truth file has 33,126 rows, one row for one image, with the columns image_name, patient_id, lesion_id, sex, age_approx, anatom_site_general_challenge, diagnosis, benign_malignant and target. There are 32,542 benign and 584 malignant images, from 2,056 patients (Fig. 5.1). The label is very skewed, it is around 56 benign images for each malignant one. Most of the benign images have the diagnosis "unknown" (27,124) or "nevus" (5,193).

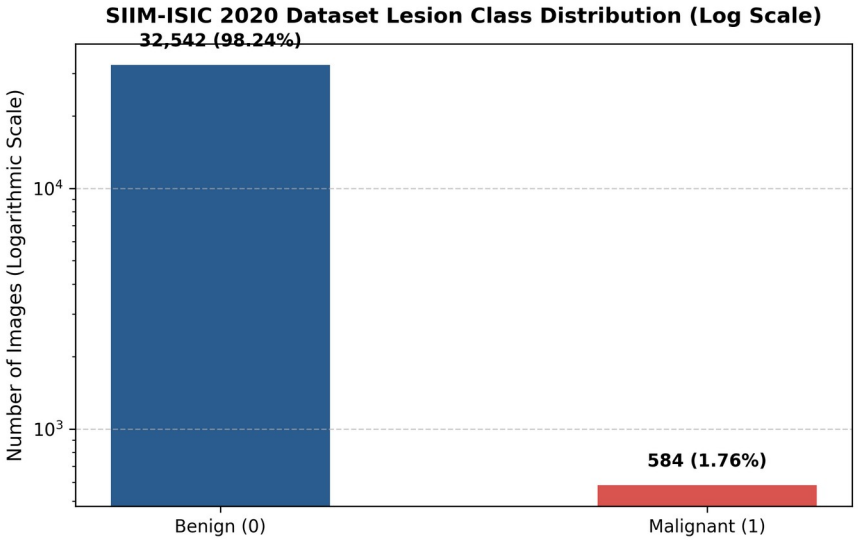


Figure 5.1: Class distribution of the ISIC 2020 training data (logarithmic scale)

We grouped the images by patient_id, and each patient was put in exactly one split. Table 5.1 and Fig. 5.2 shows the result. The baseline script repeats this check each time it runs, with three assert statements, which stop the program if any two splits share a patient. For this report we checked the three CSV files once again and found zero overlap between every pair of splits. Training ended up with 1.95% malignant images, against 1.31% in validation and 1.38% in test. The gap is small, and it is there because we moved whole patients and not single images.

Table 5.1: Patient-level split of the data

Split	Images	Benign	Malignant	Malignant %	Patients
Training	22,787	22,342	445	1.95	1,439
Validation	5,419	5,348	71	1.31	307
Test	4,920	4,852	68	1.38	310
Total	33,126	32,542	584	1.76	2,056

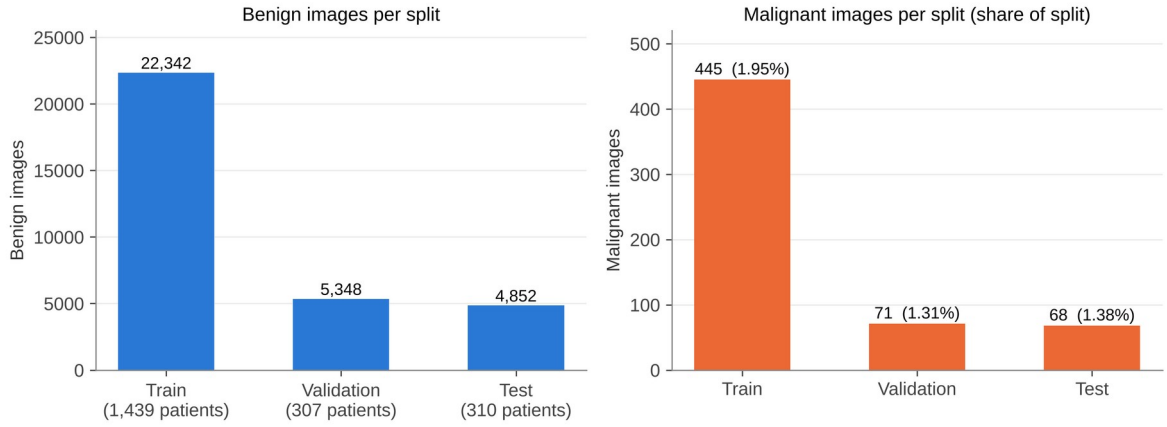


Figure 5.2: Benign and malignant images in each split

5.1.2 Preprocessing and augmentation

Our loader opens every .jpg with PIL, converts it to RGB and resizes it to 224 × 224. Then we scale the pixels to the range 0 to 1 and normalise them with the ImageNet channel mean and standard deviation. DenseNet201 learned its weights on images normalised this way, so we kept the same values. Training images also get a light random augmentation. We kept augmentation small, so that colour and texture, which carry diagnostic information, will not change too much. Validation and test images are only resized and normalised (Table 5.2).

Table 5.2: Preprocessing and augmentation settings

Step	Setting	Training	Validation / Test
Resize	224 × 224	Yes	Yes
Horizontal flip	probability 0.5	Yes	No
Rotation	random, up to ±15°, bilinear	Yes	No
Colour jitter	brightness 0.1, contrast 0.1, saturation 0.1, hue 0.05	Yes	No
Normalisation	mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225]	Yes	Yes

5.1.3 Model

Backbone is the DenseNet201 from torchvision with ImageNet-1K weights [18]. Its 1,000-class classifier is replaced with an identity layer, and so the backbone gives the 1,920-dimensional vector from global average pooling. A linear layer maps this vector into one logit z. Probability of malignancy is $\hat{p} = \sigma(z) = 1 / (1 + e^{-z})$. Listing 5.1 shows the model class from baseline script.

Listing 5.1: Model class (src/EXP_01_BASELINE/densenet201_baseline.py)

```
class DermInsightDenseNet201(nn.Module):
    def __init__(self):
        super().__init__()
        weights = DenseNet201_Weights.IMAGENET1K_V1
        backbone = models.densenet201(weights=weights)
        self.backbone = backbone
        in_features = backbone.classifier.in_features
        self.backbone.classifier = nn.Identity()
        self.head = nn.Linear(in_features, 1)

    def forward(self, x):
        features = self.backbone(x)
        logits = self.head(features)
        return logits.squeeze(1)
```

5.1.4 Experiment 1: baseline with weighted BCE

Baseline uses binary cross-entropy with logits and a weight for the positive class. This weight is taken from training counts, w+ = N_neg / N_pos = 22,342 / 445 = 50.2067, so in the loss each malignant image counts about 50 times more than a benign one:

$$L = - [\, w+ \cdot y \cdot log\ \sigma(z) + (1 - y) \cdot log(1 - \sigma(z)) \,]$$

In this first run, only the linear head was trained. Listing 5.2 shows how all the other parameters were frozen. We trained it for just 3 epochs on a CPU, with batch size 32 and AdamW [22] at learning rate 1e-4 and weight decay 1e-4. Threshold was the default 0.5. One thing to note is that this "baseline" already uses a weighted loss, so it is not a plain unweighted model.

Listing 5.2: Freezing the backbone and setting the weighted loss (Exp 1)

```
pos_weight = compute_class_weights(train_df)   # 22342 / 445 = 50.2067
model = DermInsightDenseNet201().to(DEVICE)
for name, param in model.named_parameters():
    if "head" not in name:
        param.requires_grad = False
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
optimizer = torch.optim.AdamW(model.parameters(), lr=args.learning_rate,
                              weight_decay=1e-4)
```

5.1.5 Experiment 2: fine-tuning with weighted BCE

Exp 2 loads the Exp 1 checkpoint and trains all the layers with same weighted loss. It uses batch size 64, AdamW with learning rate 1e-4 and weight decay 1e-4, and a ReduceLROnPlateau scheduler which halves the learning rate when validation ROC-AUC does not improve for one epoch. Training runs maximum 8 epochs and it stops after 3 epochs without improvement (Listing 5.3). It stopped after epoch 6, and the best checkpoint was from epoch 3. After this the threshold was chosen on validation set, as explained in Section 5.1.9.

Listing 5.3: Scheduler, model selection and early stopping (Exp 2, condensed)

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="max", factor=0.5, patience=1, min_lr=1e-6)
MAX_EPOCHS, EARLY_STOPPING_PATIENCE = 8, 3
for epoch in range(1, MAX_EPOCHS + 1):
    ...                                          # one pass over train_loader
    val_loss, val_auc, val_ap = evaluate(model, val_loader)
    scheduler.step(val_auc)
    if val_auc > best_auc:
        best_auc, best_epoch = val_auc, epoch
        torch.save({"state_dict": model.state_dict(), "epoch": epoch,
                    "auc": best_auc}, STAGE2B_MODEL)
        epochs_without_improvement = 0
    else:
        epochs_without_improvement += 1
    if epochs_without_improvement >= EARLY_STOPPING_PATIENCE:
        break
```

5.1.6 Experiment 3: WeightedRandomSampler

Exp 3 does not change the loss, it changes which data the model will see. Each training image gets a sampling weight inversely proportional to the size of its class, and the PyTorch WeightedRandomSampler draws the batches with replacement using these weights. In effect, a batch of 64 has roughly half benign and half malignant images on average. Since the sampler already balances the classes, we used plain BCE with no class weight. The rest we read from the saved checkpoint: ImageNet weights for DenseNet201, learning rate 1e-4, weight decay 1e-4 and seed 42. The run was for 5 epochs, best epoch was 2, and test threshold was 0.5.

5.1.7 Experiment 4: focal loss

Focal loss [21] multiplies the cross-entropy with a factor, which becomes small for examples that are already classified well:

$$FL(p_t) = -\alpha_t \cdot (1 - p_t)^{\gamma} \cdot log(p_t)$$

In this formula p_t is the probability the model gives to the true class. The checkpoint stores $\alpha = 0.75$ and $\gamma = 2.0$. Say a benign image already has p_t = 0.99. Its loss gets multiplied by $(1 - 0.99)^2 = 0.0001$, which is almost nothing. Our training split has 22,342 benign images and many of them are easy cases like this, so focal loss keeps them from taking over the training. The run was for 5 epochs, best epoch was 5, and test threshold was 0.5. The training script of this run is not there in project folder, so batch size and optimiser settings could not be confirmed from code.

5.1.8 Experiment 5: random oversampling

In Exp 5 we oversampled only the 445 malignant training images. The training summary says this was done with a WeightedRandomSampler on the training set. So each malignant image gets drawn many times per epoch. Validation and test were not touched, and they keep their natural ratio of about 1.3 to 1.4% malignant. Plain BCE loss and AdamW (learning rate 1e-4, weight decay 1e-4, confirmed from saved optimiser state) were used. The run was for 5 epochs and best epoch was 2. Threshold of 0.7296 was the value having best F1 score on validation set. That makes Exp 5 quite close to Exp 3, as both use the WeightedRandomSampler. What differs is the settings, and Exp 5 tunes its threshold to 0.7296 while Exp 3 stays at 0.5. We could not find the scripts of these two runs in our folder, so there may be other differences we cannot see.

5.1.9 Threshold selection

The 0.5 cut-off does not suit every model. With pos_weight 50.21, many benign images get pushed to high probabilities. Focal loss does the opposite, and in Exp 4 no malignant test image went above 0.73. In Exp 2, the evaluation script scores the validation set, tries every threshold from 0.01 to 0.99 in steps of 0.01, and keeps the one which gives highest F1 score (Listing 5.4). This gave $\tau = 0.86$. At this value, validation sensitivity was 30.99%, specificity 98.50% and F1 score 25.43%. The same rule, best F1 on validation, gave $\tau = 0.7296$ in Exp 5. Exp 1, 3 and 4 were tested at 0.5.

Listing 5.4: Threshold sweep on the validation set (stage2b_final_evaluation.py, condensed)

```
val_labels, val_probs = collect_predictions(model, val_loader)
best_threshold, best_f1 = 0.5, -1
for threshold in np.arange(0.01, 1.00, 0.01):
    preds = (val_probs >= threshold).astype(int)
    f1 = f1_score(val_labels, preds, zero_division=0)
    if f1 > best_f1:
        best_f1, best_threshold = f1, float(threshold)
# the test set is scored only after the threshold is fixed
test_labels, test_probs = collect_predictions(model, test_loader)
test_preds = (test_probs >= best_threshold).astype(int)
```

5.1.10 Evaluation metrics

From the confusion matrix counts TP, FP, TN and FN we report sensitivity = TP / (TP + FN), specificity = TN / (TN + FP), precision = TP / (TP + FP), F1 = 2 · precision · sensitivity / (precision + sensitivity) and accuracy = (TP + TN) / N. ROC-AUC and average precision (AP) are computed from probabilities, so they don't depend on threshold. AP is the average_precision_score of scikit-learn. In the Exp 2 result file this value is called "PR-AUC", but it is same AP function. Only Exp 5 saved a PR-AUC with trapezoidal rule (0.0975) also, which is little lower than its AP (0.1041). We

show these two values separately and do not mix them.

Table 5.3: Training settings of the five experiments

Setting	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5
Start weights	ImageNet	Exp 1 checkpoint	ImageNet	ImageNet	ImageNet
Trainable layers	Head only	All	All*	All*	All*
Imbalance method	W-BCE (50.21)	W-BCE (50.21)	WRS + BCE	Focal (α 0.75, γ 2)	Oversampling (WRS) + BCE
Batch size	32	64	64	32†	32†
Optimiser / LR	AdamW, 1e-4	AdamW, 1e-4 + scheduler	lr 1e-4, wd 1e-4	lr 1e-4†	AdamW, 1e-4
Epochs run / best	3 / 3	6 / 3	5 / 2	5 / 5	5 / 2
Test threshold	0.5	0.86 (val)	0.5	0.5	0.7296 (val)

* Assumed from the 70 MB full-model checkpoints, as training scripts are not available. † Taken from experiment comparison sheet in the project folder, and not from code or checkpoints.

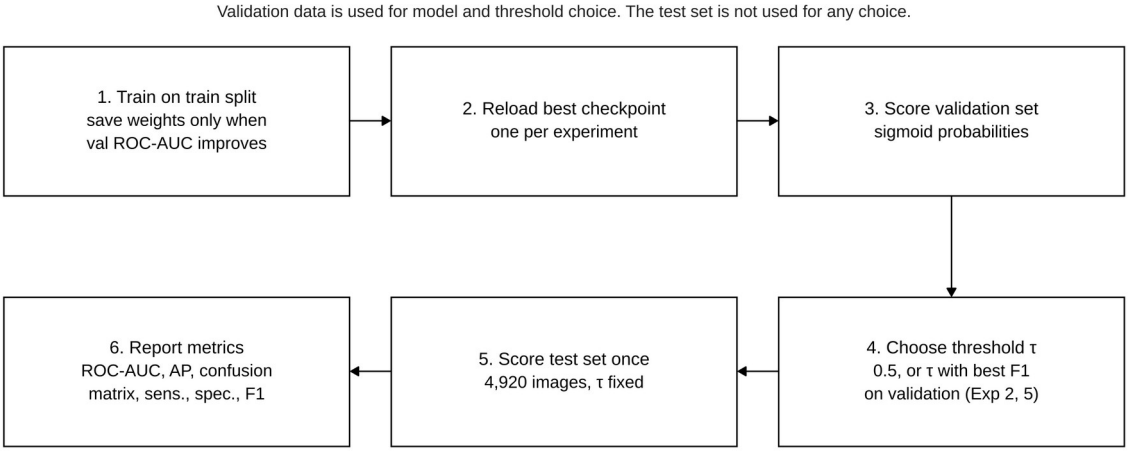


Figure 5.3: Evaluation protocol followed in every experiment

5.2 Testing

There is no formal unit test suite for the pipeline. Testing was done by the checks which are built into the scripts, and by a separate verification of all the reported numbers for this report. Table 5.4 lists these checks and what we got.

Table 5.4: Checks done on the pipeline and the results

Check	How it was done	Result
Patient overlap	assert statements in train_model(); repeated on the three CSV files	0 shared patients
Split counts	Counted rows, labels and patients in each CSV	Match Table 5.1
Image files present	Exp 2 script checks each train and validation image path before the training	No missing files reported
Forward pass	--sanity-check option runs 4 images through the model and prints shapes, probabilities and loss	Available in baseline script
Test metrics, Exp 3–5	ROC-AUC, AP and confusion matrix computed again from saved prediction CSVs	Match the saved metric files
Test metrics, Exp 1–2	Sensitivity, specificity, precision, F1, accuracy computed again from saved confusion counts	Match; per-image predictions not saved
Checkpoint contents	Read the epoch, best AUC and settings which are stored in Exp 3–5 checkpoints	Match the history files

Check	How it was done	Result
Threshold file of Exp 2	Compared stage2b_validation_threshold_analysis.csv with the evaluation JSON	Do not match; see Section 7.6

The last row needs some attention. The threshold-analysis CSV saved for Exp 2, and the threshold plots made from it, do not agree with validation numbers inside the final evaluation file. For example at $\tau = 0.86$, the CSV gives 5 true positives on validation, but the evaluation file gives 22. We trust the evaluation file more. It came from the same script that picked $\tau = 0.86$ and then scored the test set. So we treat it as the correct record, and we left the CSV-based plots out of this report.

CHAPTER 6

6. PROJECT DEMONSTRATION

There is no app or GUI in Project-I. To demo DermInsight AI we run the Python scripts from the command line, for example densenet201_baseline.py, and then open the JSON files and plots they write.

6.1 Running an Experiment

Baseline script takes its settings from command line. Using its own file location, it works out the project root. Then it reads the three split CSVs from data/processed and the images from data/raw/train_images. We used the command in Listing 6.1 for the recorded baseline run. With --sanity-check, the script first sends four images through the untrained model and prints the tensor shapes, probabilities and loss. We used this to catch path or shape mistakes before starting a long run.

Listing 6.1: Command for the baseline run

```
$ python src/EXP_01_BASELINE/densenet201_baseline.py \
      --epochs 3 --batch-size 32 --learning-rate 1e-4 --sanity-check
```

After the last epoch the script loads the best weights back. It scores the 4,920 test images at $\tau = 0.5$, then saves four plots and two JSON files. Listing 6.2 is a part of the test metrics JSON from Exp 1. We only rounded the values to four decimals.

Listing 6.2: Part of results/EXP_01_BASELINE/metrics/densenet201_test_metrics.json

```
{
  "test": {
    "accuracy": 0.4258, "precision": 0.0215, "recall": 0.9118,
    "specificity": 0.4190, "f1": 0.0420,
    "roc_auc": 0.8000, "average_precision": 0.0626
  },
  "test_loss": 1.0801
}
```

6.2 Threshold Selection and Final Test Run

For Exp 2 the evaluation is done by a separate script, and this keeps the test set away from all the choices made while training. It loads best checkpoint, chooses the threshold on validation set and only after that scores the test set. Listing 6.3 shows the test block of its output file.

Listing 6.3: Test block of results/EXP_02_WEIGHTED_BCE/final_evaluation/stage2b_test_evaluation.json

```
"test": {
  "roc_auc": 0.8298, "pr_auc": 0.1288, "threshold": 0.86,
  "sensitivity": 0.2206, "specificity": 0.9831, "precision": 0.1546,
  "f1": 0.1818, "accuracy": 0.9726,
  "tn": 4770, "fp": 82, "fn": 53, "tp": 15
},
"test_samples": 4920, "test_positive": 68, "test_negative": 4852
```

6.3 Output Files

Each experiment writes its outputs in its own folder (Table 6.1). Exp 3, 4 and 5 also saved probability of each test image, and because of this we could recompute their metrics and draw the combined ROC and PR curves in Chapter 7. Fig. 6.1 is one such saved output, the histogram of predicted probabilities on test set for Exp 3. Nearly all the benign images are near 0, and the malignant images are spread over the full range.

Table 6.1: Output files of each experiment

Exp	Checkpoint	Main result files	Plots
1	densenet201_baseline_best.pt	baseline metrics JSON, test metrics JSON	loss, ROC, PR, confusion matrix
2	densenet201_stage2b_weighted_bce.pt	training metrics JSON, test evaluation JSON, threshold CSV	confusion matrix, threshold plots
3	densenet201_weighted_sampler_best.pt	history JSON, test metrics JSON, test predictions CSV	loss, val ROC-AUC, ROC, PR, CM, probabilities
4	densenet201_focal_loss_best.pt	history JSON, test metrics JSON, test predictions CSV	ROC, PR, CM, probabilities
5	densenet201_random_oversampling_best.pt	history and summary JSON, test metrics JSON/CSV, predictions CSV	ROC, PR, confusion matrix

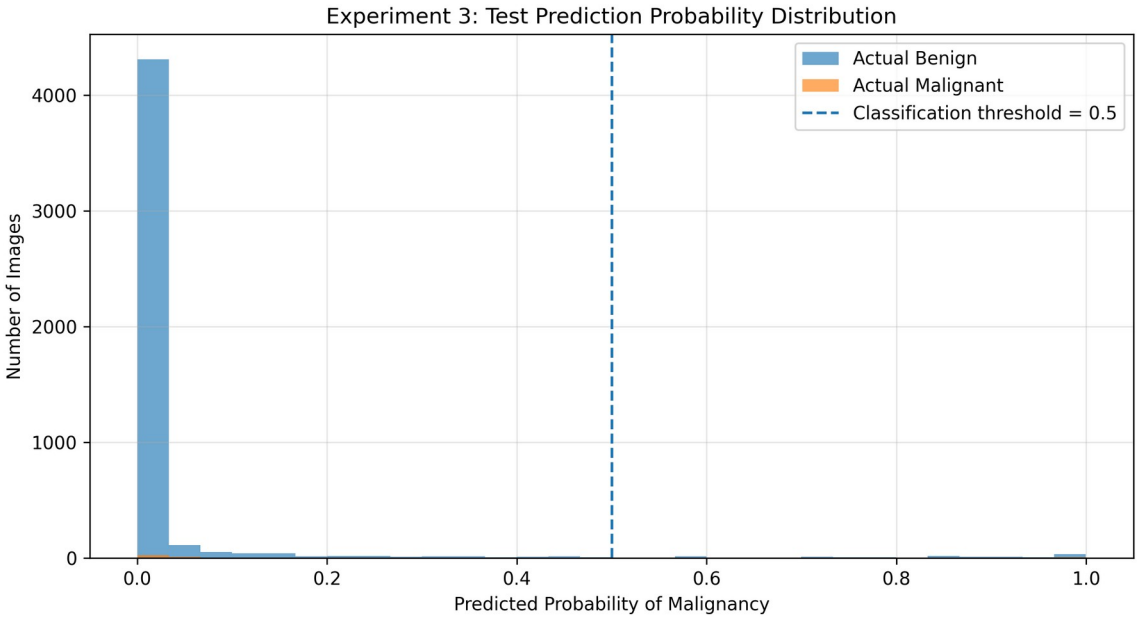


Figure 6.1: Predicted probabilities on the test set, Exp 3 (saved output)

CHAPTER 7

7. RESULT AND DISCUSSION (COST ANALYSIS)

All test results in this chapter are on the same 4,920 test images from 310 patients (4,852 benign and 68 malignant). None of these patients was used for training or for choosing any epoch or threshold. Validation numbers are always marked as validation.

7.1 Training and Validation Behaviour

Table 7.1 and Fig. 7.1 show validation ROC-AUC of each run. The four runs after baseline got a clearly higher validation ROC-AUC (0.906 to 0.919) than Exp 1 (0.847), where only the head was trained. Exp 2 and Exp 4 stayed fairly stable over epochs. Exp 3 and Exp 5 had peak at epoch 2 and then dropped, while their training loss kept on falling, and this suggests the models started overfitting the repeated malignant images. Since the checkpoint was chosen by validation ROC-AUC, the final models were protected from these later epochs.

Table 7.1: Validation results of the five experiments

Exp	Strategy	Epochs run	Best epoch	Val ROC-AUC	Val AP
1	Baseline, frozen backbone, W-BCE	3	3	0.8472	0.0811
2	W-BCE, full fine-tuning	6	3	0.9188	0.1747
3	WeightedRandomSampler	5	2	0.9062	0.1894
4	Focal loss	5	5	0.9181	0.1800
5	Random oversampling	5	2	0.9082	0.2176

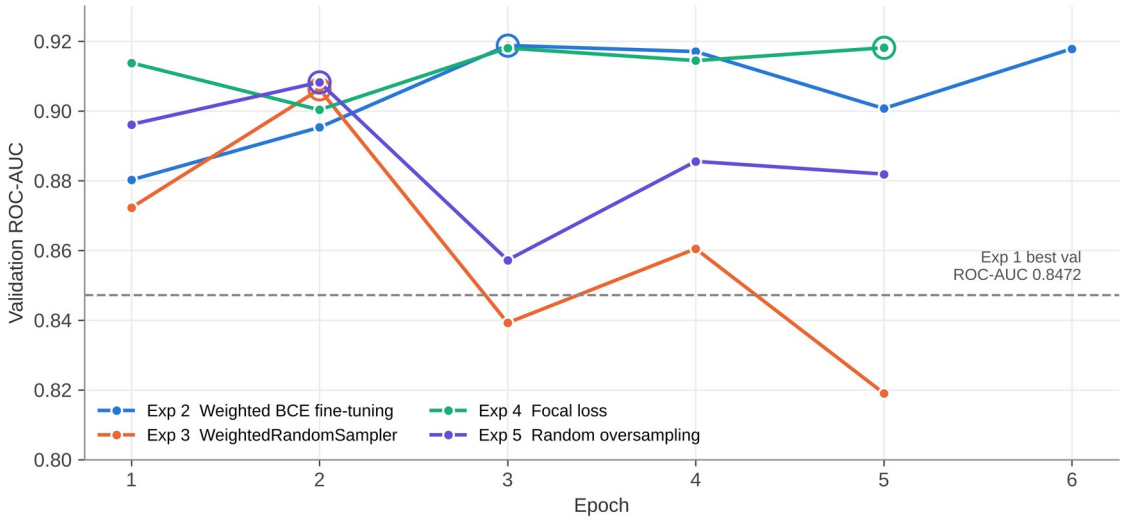


Figure 7.1: Validation ROC-AUC by epoch (circles mark the saved checkpoint)

Fig. 7.2 shows the loss curves of baseline. Both losses come down steadily over three epochs, and validation loss is still falling at the end, so the frozen-backbone model was still learning when training was stopped. As only the head was trainable and it was three epochs on CPU, Exp 1 is better seen as a starting point.

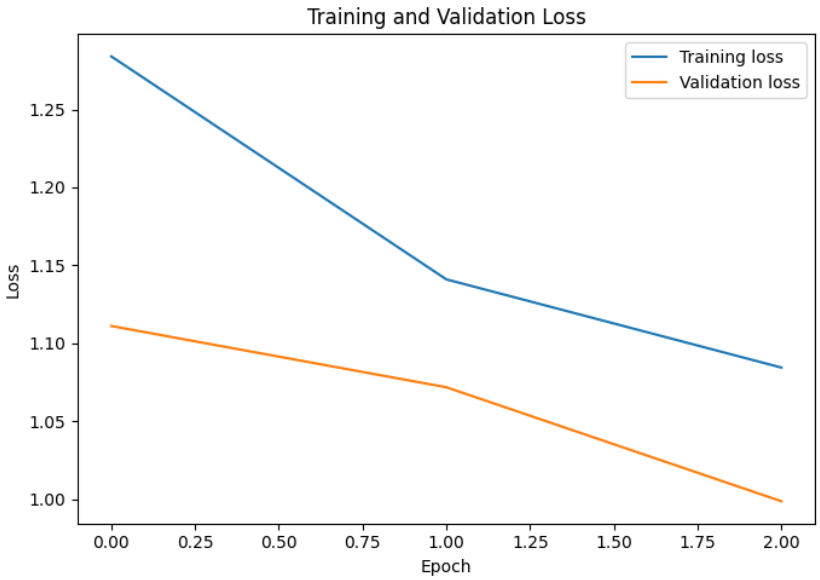


Figure 7.2: Training and validation loss of Exp 1 (x-axis: epoch index 0 to 2)

7.2 Test Results

Table 7.2 gives full test results and Fig. 7.3 shows the confusion matrices. Ranking metrics are moderate for all runs. Test ROC-AUC is between 0.800 (Exp 1) and 0.871 (Exp 5), and AP is between 0.063 and 0.140, where the base rate is 0.014. At the chosen threshold the results differ a lot more. Exp 1 finds 62 out of 68 cancers, but it also flags 2,819 benign images. Exp 2, 4 and 5 keep false positives below 90, but they find only 10 to 15 cancers. Exp 3 comes in between, with 25 cancers found and 153 false positives.

Table 7.2: Results on the test set (N = 4,920; 68 malignant)

Exp	τ	ROC-AUC	AP	Sens. %	Spec. %	Prec. %	F1 %	Acc. %	TN / FP / FN / TP
1	0.50	0.8000	0.0626	91.18	41.90	2.15	4.20	42.58	2033 / 2819 / 6 / 62
2	0.86	0.8298	0.1288	22.06	98.31	15.46	18.18	97.26	4770 / 82 / 53 / 15
3	0.50	0.8218	0.1286	36.76	96.85	14.04	20.33	96.02	4699 / 153 / 43 / 25
4	0.50	0.8601	0.1402	14.71	99.24	21.28	17.39	98.07	4815 / 37 / 58 / 10
5	0.7296	0.8708	0.1041	19.12	98.25	13.27	15.66	97.15	4767 / 85 / 55 / 13

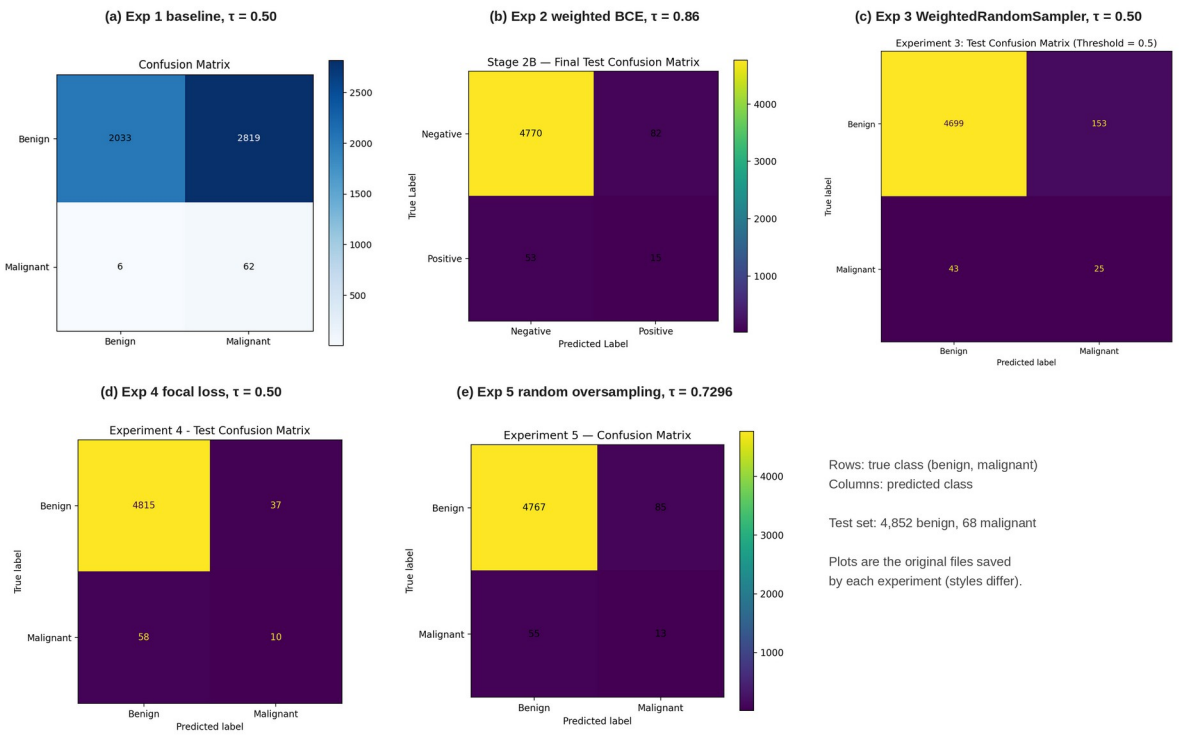


Figure 7.3: Test confusion matrices of the five experiments (original saved plots)

Fig. 7.4 and Fig. 7.5 compare the runs over all thresholds. Curves for Exp 3, 4 and 5 are drawn from their saved test predictions. Exp 1 and Exp 2 did not save per-image predictions, so they are shown only as single operating points. In ROC plot the three curves are near to each other, and Exp 5 is little higher for most of the range. In PR plot, precision drops fast for all models once recall goes above about 0.1, and focal loss (Exp 4) keeps highest precision at low recall. All curves stay well above 1.38% base rate, but none of them reach a region where both recall and precision are high.

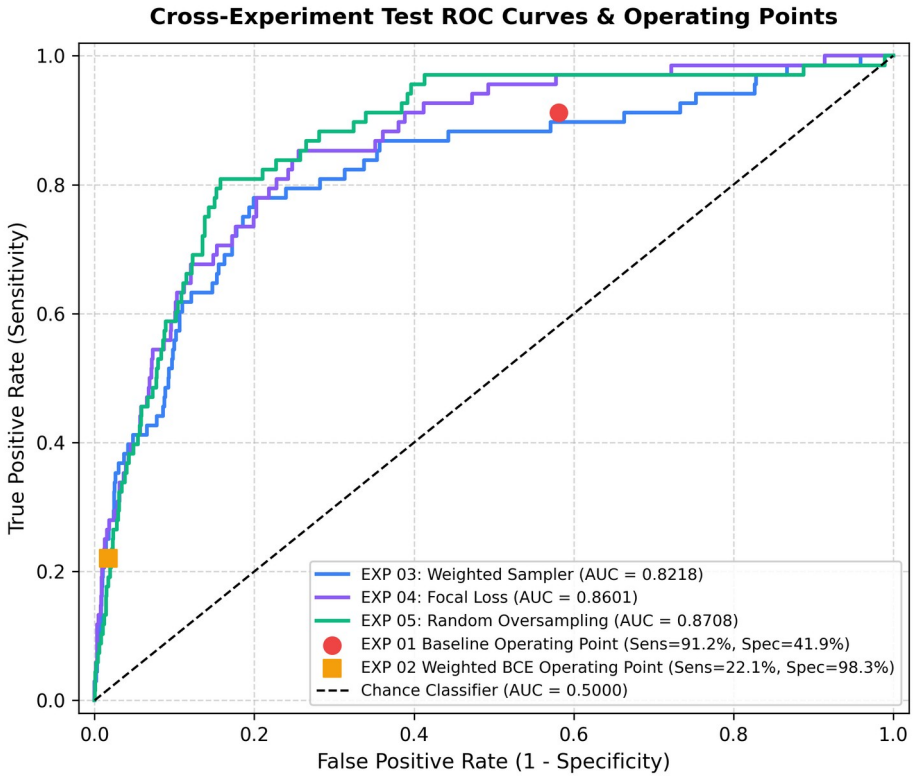


Figure 7.4: Test ROC curves (Exp 3–5) and operating points (Exp 1–2)

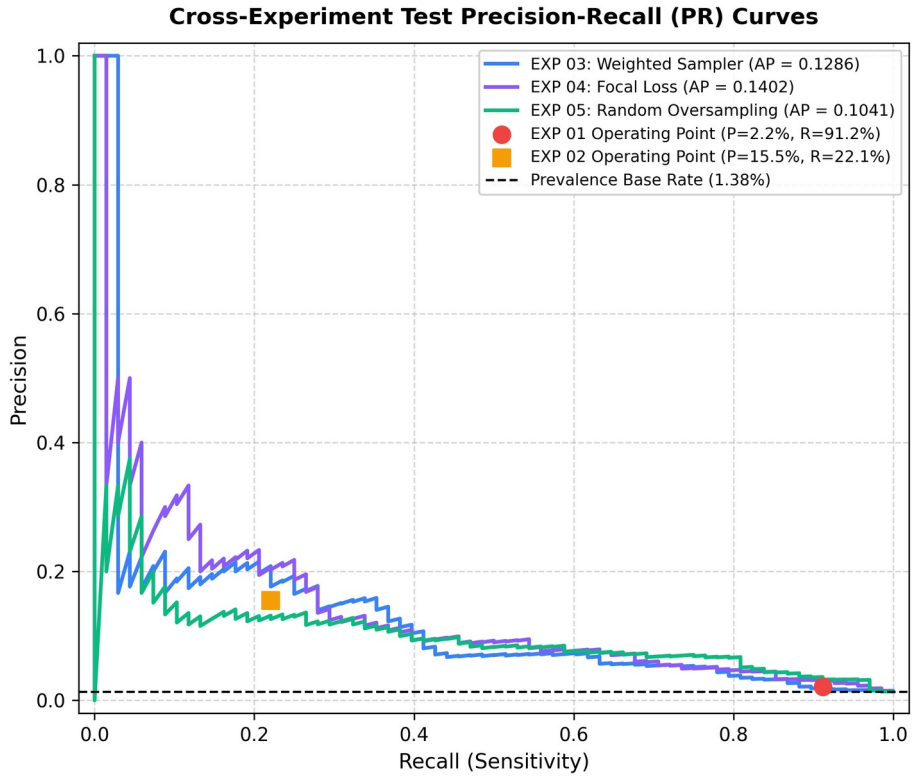


Figure 7.5: Test precision-recall curves (Exp 3–5) and operating points (Exp 1–2)

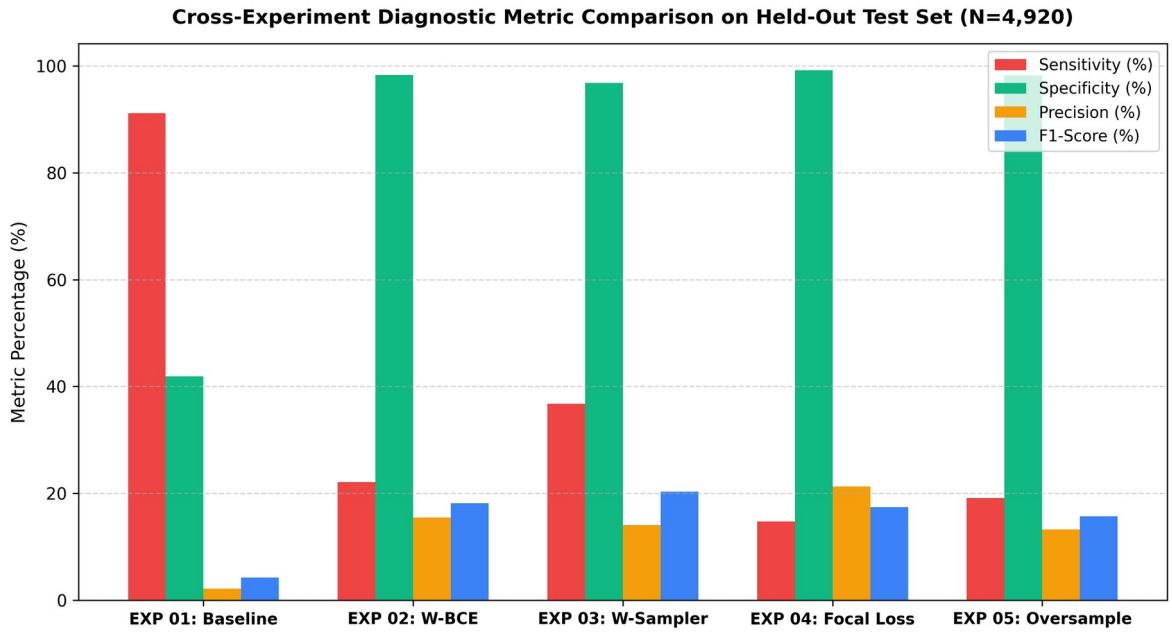
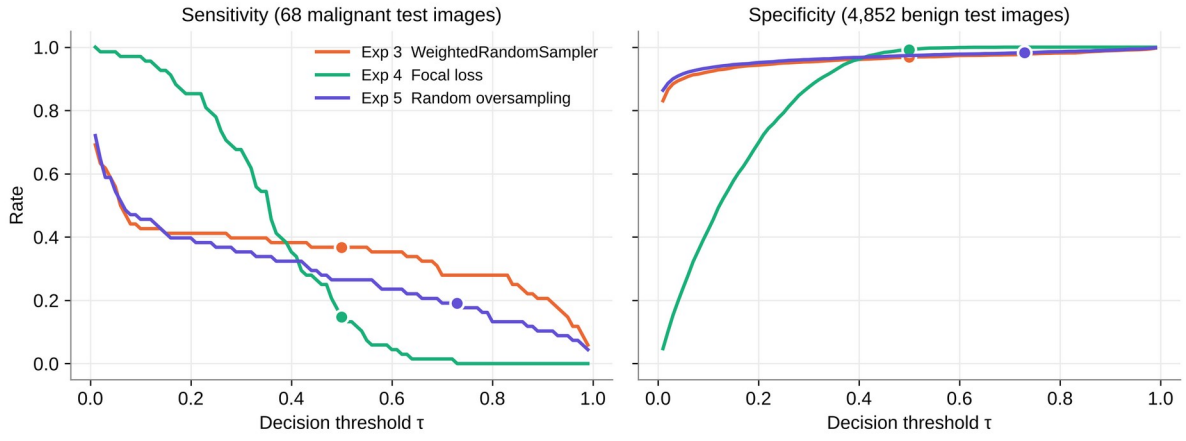


Figure 7.6: Sensitivity, specificity, precision and F1 of each experiment on the test set

7.3 Effect of the Decision Threshold

Fig. 7.7 shows how sensitivity and specificity changes with τ for three runs whose test predictions were saved. This figure only describes the models, and no threshold was chosen from it. First, Exp 3 and Exp 5 push most of the benign images very close to 0 (median benign probability 0.0005 and almost 0), so their specificity stays above 90% even at low thresholds, but sensitivity curve is flat between about 0.1 and 0.6. Exp 4 is different, because focal loss squeezes all its scores into a narrow range. In Exp 4, no malignant test image got a probability more than 0.73, and median malignant score was 0.36, so the default 0.5 cut-off misses most of the cancers. At τ = 0.3 the same model will find 46 of the 68 cancers with specificity of 87.3%.



Dots mark the threshold actually used for each experiment. Curves are drawn from the saved test predictions for illustration only; no threshold was chosen on the test set.

Figure 7.7: Sensitivity and specificity against the threshold on the test set (descriptive only)

Table 7.3 shows the same effect for Exp 5 at two thresholds. The threshold chosen on validation (0.7296) is stricter than 0.5. On test set it removes 45 false positives, but it also loses 5 out of the 18 cancers which 0.5 would have found. Since there are only 71 malignant validation images, the best-F1 threshold moves a lot with just a few images, and in this run it did not carry over well to the test set.

Table 7.3: Exp 5 on the test set at two thresholds

τ	TP	FP	FN	Sens. %	Spec. %	Prec. %	F1 %
0.50 (default)	18	130	50	26.47	97.32	12.16	16.67
0.7296 (best F1 on validation)	13	85	55	19.12	98.25	13.27	15.66

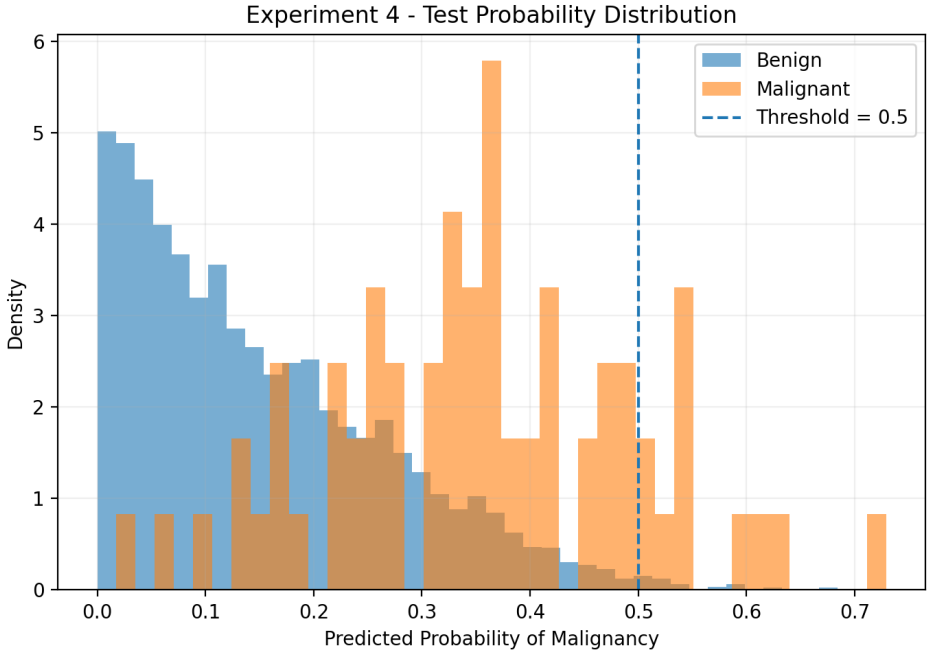


Figure 7.8: Predicted probabilities on the test set, Exp 4 (focal loss)

7.4 Accuracy and Uncertainty

Here accuracy is not a good guide. Look at Exp 4. It has 98.07% accuracy, the highest of all, and yet 58 of the 68 cancers were missed. Exp 1 is at the bottom with 42.58%, and it missed only 6. A model which always predicts benign would get 98.62% on this test set. So we compare the models by sensitivity, specificity and confusion matrix, and accuracy is used only as a side number.

Positive side of the test set is small. Table 7.4 gives 95% Wilson intervals [31] for sensitivity of each run. The intervals are wide, around ±10 percentage points, and intervals of Exp 2, 4 and 5 overlap a lot. So we can say Exp 1 and Exp 3 found more cancers than the high-specificity runs, but we cannot rank Exp 2, 4 and 5 among themselves with confidence.

Table 7.4: Test sensitivity with 95% Wilson intervals (68 malignant images)

Exp	Cancers found	Sensitivity %	95% interval %
1	62 / 68	91.18	82.1 – 95.9
2	15 / 68	22.06	13.8 – 33.3
3	25 / 68	36.76	26.3 – 48.6
4	10 / 68	14.71	8.2 – 25.0
5	13 / 68	19.12	11.5 – 30.0

7.5 Comparison of the Strategies

- **Loss weighting (Exp 1, Exp 2).** When the backbone was frozen and τ = 0.5, the weight of 50.21 pushed most of the scores above 0.5, which gave high sensitivity and very low specificity. Full fine-tuning raised validation ROC-AUC from 0.847 to 0.919. After the

threshold was moved to 0.86 on validation, the test result became opposite, with high specificity and low sensitivity.

- **Weighted sampling (Exp 3).** This gave best balance at the default threshold and the highest F1 (20.33%), though its test ROC-AUC (0.822) was not the best. Balanced batches let the model see malignant images often, without very big loss weights.
- **Focal loss (Exp 4).** It gave best AP (0.140) and the highest precision, but at $\tau = 0.5$ it was too conservative, as the loss pushed the scores into a narrow low range.
- **Random oversampling (Exp 5).** It gave the highest test ROC-AUC (0.871), so it ranked the images best. But the threshold chosen on validation did not transfer well and final sensitivity was only 19.12%.

No single strategy was best in every measure. Imbalance methods mostly improved the ranking of images, and the final balance between cancers found and false alarms was decided more by the threshold than by the method. Exp 3 and Exp 5 both use sampling, but they differ in batch size, threshold and may be other settings which we could not confirm, so we are not making a strong conclusion about sampling versus loss weighting.

7.6 Discussion

Where the results fall short. Even the best run misses more than 60% of the cancers at a usable false-alarm rate, and for a screening aid this is too low. Test ROC-AUC values (0.80 to 0.87) are also lower than validation values (0.85 to 0.92). As the model and threshold were chosen using 71 malignant validation images, part of this drop is likely due to selection on a small set.

Incomplete work. In our earlier notes an Experiment 6 is mentioned, but there is no script, checkpoint, log or prediction file for it in the project folder. As far as we know the run was not completed, so we do not report it. The folder results/EXP_01_BASELINE/gradcam is empty and no Grad-CAM code is there, so explainability is future work.

Problems found while checking the results. Exp 2 threshold-analysis CSV and its plots do not match the final evaluation file (Section 5.2), so we did not use them. Training scripts for Exp 3, 4 and 5 are missing, so some of their settings come only from comparison sheet (marked in Table 5.3). Exp 1 and Exp 2 did not save per-image predictions. Library versions were not recorded. The duplicate images listed for ISIC 2020 (425 according to our earlier audit notes) were not removed before splitting, and duplicates of the same lesion can still affect results inside one patient's images [6].

Limits of the data. All images are from one public dataset. ISIC 2020 has no Fitzpatrick skin type labels, and public dermatology image sets have very few images of darker skin [10, 14], so we cannot say how these models work on darker skin. The model uses only images, while doctors also use age, sex and body site. The probabilities are not calibrated [15], so a score of 0.7 should not be taken as a 70% risk.

7.7 Cost Analysis

There was no licence or equipment cost for the project (Table 7.5). Main resource was GPU time on free cloud sessions. For example, training history of Exp 3 records about 10 minutes per epoch, or about 51 minutes in total for its 5 epochs.

Table 7.5: Cost of the project

Item	Cost	Note
ISIC 2020 data	Free	Public dataset, used under its licence for research
Software (Python, PyTorch, scikit-learn, etc.)	Free	Open source
GPU training (Google Colab)	Free tier	Session limits made long runs difficult
Storage (Google Drive, laptop)	Existing	About 0.5 GB of checkpoints
Total direct cost	₹ 0	Team time not counted

Errors of a screening model also have a cost. A false negative can delay the treatment of a cancer, and a false positive leads to one extra visit and maybe a biopsy. We don't have real cost data, so Table 7.6 is only an illustration using assumed relative costs. One false positive is taken as 1 unit, and one false negative as 10, 50 or 100 units. The 50:1 ratio is near to the class weight used in training.

Table 7.6: Illustrative error cost on the test set (assumed unit costs, not real prices)

Exp	FP	FN	Cost (FN = 10)	Cost (FN = 50)	Cost (FN = 100)
1	2,819	6	2,879	3,119	3,419
2	82	53	612	2,732	5,382
3	153	43	583	2,303	4,453
4	37	58	617	2,937	5,837
5	85	55	635	2,835	5,585

In all three assumptions Exp 3 has lowest cost, since it misses the fewest cancers among those runs which do not flood the clinic with false positives. Exp 1 becomes competitive only when one missed cancer is valued at 100 false alarms or more. If the ratio changes, all these costs change too. We put them here for one reason, to show that a missed cancer has to count for much more than a false alarm.

CHAPTER 8

8. CONCLUSION

DermInsight AI, the pipeline we built, looks only at the dermoscopic image and says benign or malignant. We tested it on SIIM-ISIC 2020, where just 1.76% of the images are malignant. The 33,126 images were split patient wise into training, validation and test sets with no shared patients, and this was checked in code. Five DenseNet201 experiments compared loss weighting, weighted sampling, focal loss and random oversampling under one protocol, where model and threshold choices were made on validation data and each model was tested only once.

Imbalance methods improved validation ROC-AUC from 0.847 to between 0.906 and 0.919, and best test ROC-AUC was 0.871. At the thresholds which were actually used, the models either found most of the cancers with very many false alarms (Exp 1), or kept false alarms low and found only 10 to 25 of the 68 cancers. The WeightedRandomSampler run gave the best balance (36.76% sensitivity at 96.85% specificity). Main lesson for us is that a good ROC-AUC and a high accuracy both can hide a big number of missed cancers, so confusion matrix and threshold should always be reported. With only 68 malignant test images, the differences between most of the runs are inside the uncertainty. DermInsight AI is a research prototype and it is not a diagnostic tool.

Future work. Following steps are planned for Project-II:

- Save per-image predictions and full settings for each run, pin the library versions, and add the missing Exp 3–5 scripts to the repository.
- Choose thresholds with a sensitivity target (like at least 80% on validation) in place of best F1, and use patient-wise cross-validation so that the threshold becomes more stable.
- Add patient metadata (age, sex, body site) to the image features and test if it improves sensitivity [23, 20].
- Add Grad-CAM [28] heatmaps and check if the model is looking at the lesion and not at rulers, hair or borders.
- Calibrate the probabilities, for example with temperature scaling [15], and test on another dataset like HAM10000 [29] to measure how well the models generalise.
- Remove duplicate images before splitting, following Cassidy et al. [6], and repeat the main experiments.

CHAPTER 9

9. REFERENCES

1. Adegun, A. and Viriri, S., 2021. Deep learning techniques for skin lesion analysis and melanoma cancer detection: a survey of state-of-the-art. *Artificial Intelligence Review*, 54(2), pp.811–841.
2. American Cancer Society, 2026. *Key statistics for melanoma skin cancer*. [online] Available at: https://www.cancer.org/cancer/types/melanoma-skin-cancer/about/key-statistics.html [Accessed 27 October 2026].
3. American Cancer Society, 2026. *Survival rates for melanoma skin cancer by stage*. [online] Available at: https://www.cancer.org/cancer/types/melanoma-skin-cancer/detection-diagnosis-staging/survival-rates-for-melanoma-skin-cancer-by-stage.html [Accessed 27 October 2026].
4. Brinker, T.J., Hekler, A., Enk, A.H. et al., 2019. Deep learning outperformed 136 of 157 dermatologists in a head-to-head dermoscopic melanoma image classification task. *European Journal of Cancer*, 113, pp.47–54.
5. Buda, M., Maki, A. and Mazurowski, M.A., 2018. A systematic study of the class imbalance problem in convolutional neural networks. *Neural Networks*, 106, pp.249–259.
6. Cassidy, B., Kendrick, C., Brodzicki, A., Jaworek-Korjakowska, J. and Yap, M.H., 2022. Analysis of the ISIC image datasets: usage, benchmarks and recommendations. *Medical Image Analysis*, 75, p.102305.
7. Celebi, M.E., Codella, N. and Halpern, A., 2019. Dermoscopy image analysis: overview and future directions. *IEEE Journal of Biomedical and Health Informatics*, 23(2), pp.474–478.
8. Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P., 2002. SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, pp.321–357.
9. Codella, N.C.F., Gutman, D., Celebi, M.E. et al., 2018. Skin lesion analysis toward melanoma detection: a challenge at the 2017 International Symposium on Biomedical Imaging (ISBI), hosted by the International Skin Imaging Collaboration (ISIC). In: *IEEE 15th International Symposium on Biomedical Imaging*, pp.168–172.
10. Daneshjou, R., Vodrahalli, K., Novoa, R.A. et al., 2022. Disparities in dermatology AI performance on a diverse, curated clinical image set. *Science Advances*, 8(32), p.eabq6147.
11. Davis, J. and Goadrich, M., 2006. The relationship between precision-recall and ROC curves. In: *Proceedings of the 23rd International Conference on Machine Learning*, pp.233–240.
12. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. and Fei-Fei, L., 2009. ImageNet: a large-scale hierarchical image database. In: *IEEE Conference on Computer Vision and Pattern Recognition*, pp.248–255.
13. Esteva, A., Kuprel, B., Novoa, R.A., Ko, J., Swetter, S.M., Blau, H.M. and Thrun, S.,

2017. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639), pp.115–118.

14. Groh, M., Harris, C., Soenksen, L. et al., 2021. Evaluating deep neural networks trained on clinical images in dermatology with the Fitzpatrick 17k dataset. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp.1820–1828.
15. Guo, C., Pleiss, G., Sun, Y. and Weinberger, K.Q., 2017. On calibration of modern neural networks. In: *Proceedings of the 34th International Conference on Machine Learning*, pp.1321–1330.
16. Haenssle, H.A., Fink, C., Schneiderbauer, R. et al., 2018. Man against machine: diagnostic performance of a deep learning convolutional neural network for dermoscopic melanoma recognition in comparison to 58 dermatologists. *Annals of Oncology*, 29(8), pp.1836–1842.
17. He, K., Zhang, X., Ren, S. and Sun, J., 2016. Deep residual learning for image recognition. In: *IEEE Conference on Computer Vision and Pattern Recognition*, pp.770–778.
18. Huang, G., Liu, Z., van der Maaten, L. and Weinberger, K.Q., 2017. Densely connected convolutional networks. In: *IEEE Conference on Computer Vision and Pattern Recognition*, pp.4700–4708.
19. Johnson, J.M. and Khoshgoftaar, T.M., 2019. Survey on deep learning with class imbalance. *Journal of Big Data*, 6(1), p.27.
20. Kawahara, J., Daneshvar, S., Argenziano, G. and Hamarneh, G., 2019. Seven-point checklist and skin lesion classification using multitask multimodal neural nets. *IEEE Journal of Biomedical and Health Informatics*, 23(2), pp.538–546.
21. Lin, T.-Y., Goyal, P., Girshick, R., He, K. and Dollár, P., 2017. Focal loss for dense object detection. In: *IEEE International Conference on Computer Vision*, pp.2980–2988.
22. Loshchilov, I. and Hutter, F., 2019. Decoupled weight decay regularization. In: *International Conference on Learning Representations.*
23. Pacheco, A.G.C. and Krohling, R.A., 2021. An attention-based mechanism to combine images and metadata in deep learning models applied to skin cancer classification. *IEEE Journal of Biomedical and Health Informatics*, 25(9), pp.3554–3563.
24. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G. et al., 2019. PyTorch: an imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems 32*, pp.8024–8035.
25. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O. et al., 2011. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, pp.2825–2830.
26. Rotemberg, V., Kurtansky, N., Betz-Stablein, B. et al., 2021. A patient-centric dataset of images and metadata for identifying melanomas using clinical context. *Scientific Data*, 8(1), p.34.
27. Saito, T. and Rehmsmeier, M., 2015. The precision-recall plot is more informative than

the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), p.e0118432.

28. Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D., 2017. Grad-CAM: visual explanations from deep networks via gradient-based localization. In: *IEEE International Conference on Computer Vision*, pp.618–626.
29. Tschandl, P., Rosendahl, C. and Kittler, H., 2018. The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. *Scientific Data*, 5, p.180161.
30. Tschandl, P., Codella, N., Akay, B.N. et al., 2019. Comparison of the accuracy of human readers versus machine-learning algorithms for pigmented skin lesion classification: an open, web-based, international, diagnostic study. *The Lancet Oncology*, 20(7), pp.938–947.
31. Wilson, E.B., 1927. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158), pp.209–212.
32. Yosinski, J., Clune, J., Bengio, Y. and Lipson, H., 2014. How transferable are features in deep neural networks? In: *Advances in Neural Information Processing Systems 27*, pp.3320–3328.

APPENDIX A – SAMPLE CODE

A.1 Dataset, transforms and metrics (src/EXP_01_BASELINE/densenet201_baseline.py)

Below functions are copied from the baseline script, only the blank lines are removed.

```
class DermImageDataset(Dataset):
    def __init__(self, dataframe: pd.DataFrame, image_dir: Path, transform=None):
        self.df = dataframe.reset_index(drop=True)
        self.image_dir = Path(image_dir)
        self.transform = transform
    def __len__(self) -> int:
        return len(self.df)
    def __getitem__(self, idx: int):
        row = self.df.iloc[idx]
        image_name = str(row["image_name"]).strip()
        image_path = self.image_dir / f"{image_name}.jpg"
        if not image_path.exists():
            raise FileNotFoundError(f"Missing image file: {image_path}")
        image = Image.open(image_path).convert("RGB")
        if self.transform is not None:
            image = self.transform(image)
        label = float(row["target"])
        return image, label, image_name

def build_transform(is_train: bool = False):
    if is_train:
        return transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomRotation(15, interpolation=InterpolationMode.BILINEAR),
            transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1,
hue=0.05),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225])])
    return transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])

def compute_class_weights(train_df: pd.DataFrame) -> torch.Tensor:
    pos_count = int(train_df["target"].sum())
    neg_count = int(len(train_df) - pos_count)
    if pos_count == 0 or neg_count == 0:
        raise ValueError("Each training split must contain both classes.")
    pos_weight = neg_count / pos_count
    return torch.tensor([pos_weight], dtype=torch.float32, device=DEVICE)

def calculate_binary_metrics(y_true, y_prob, threshold: float = 0.5):
    y_pred = (y_prob >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    specificity = tn / (tn + fp) if (tn + fp) > 0 else 0.0
    has_both_classes = np.unique(y_true).size > 1
    roc_auc = roc_auc_score(y_true, y_prob) if has_both_classes else float("nan")
    avg_precision = average_precision_score(y_true, y_prob) if has_both_classes else
float("nan")
    return {"accuracy": accuracy_score(y_true, y_pred),
            "precision": precision_score(y_true, y_pred, zero_division=0),
            "recall": recall_score(y_true, y_pred, zero_division=0),
            "specificity": specificity,
            "f1": f1_score(y_true, y_pred, zero_division=0),
            "roc_auc": roc_auc, "average_precision": avg_precision}
```

A.2 Patient overlap check and training loop (densenet201_baseline.py, train_model)

These lines are copied from train_model(), and the progress print inside the batch loop is left out.

```
train_patients = set(train_df["patient_id"].astype(str))
val_patients = set(val_df["patient_id"].astype(str))
test_patients = set(test_df["patient_id"].astype(str))
assert train_patients.isdisjoint(val_patients), "Patient overlap between train and
validation"
assert train_patients.isdisjoint(test_patients), "Patient overlap between train and
test"
assert val_patients.isdisjoint(test_patients), "Patient overlap between validation
and test"

for epoch in range(args.epochs):
    model.train()
    epoch_loss = 0.0
    for batch_idx, (images, labels, _) in enumerate(train_loader):
        images = images.to(DEVICE)
        labels = labels.to(DEVICE).float()
        optimizer.zero_grad()
        logits = model(images)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * images.size(0)
    train_loss = epoch_loss / len(train_df)
    train_losses.append(train_loss)
    val_loss, metrics, y_true, y_prob = evaluate_model(model, val_loader, criterion,
DEVICE)
    val_losses.append(val_loss)
    if np.isfinite(metrics["roc_auc"]) and metrics["roc_auc"] > best_auc:
        best_auc = metrics["roc_auc"]
        best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
        torch.save({"state_dict": best_state, "epoch": epoch + 1, "auc": best_auc},
                   models_dir / "densenet201_baseline_best.pt")
```

A.3 Validation threshold sweep and test evaluation (src/EXP_02_WEIGHTED_BCE/stage2b_final_evaluation.py)

In the original file each argument is written in a separate line. Here it is shown in a shorter layout, and the logic is same.

```
thresholds = np.arange(0.01, 1.00, 0.01)
best_threshold, best_f1 = 0.5, -1
threshold_results = []
for threshold in thresholds:
    preds = (val_probs >= threshold).astype(int)
    precision = precision_score(val_labels, preds, zero_division=0)
    sensitivity = recall_score(val_labels, preds, zero_division=0)
    specificity = (((val_labels == 0) & (preds == 0)).sum()
                   / max((val_labels == 0).sum(), 1))
    f1 = f1_score(val_labels, preds, zero_division=0)
    threshold_results.append({"threshold": float(threshold), "sensitivity":
float(sensitivity),
                              "specificity": float(specificity), "precision":
float(precision),
                              "f1": float(f1)})
    if f1 > best_f1:
        best_f1 = f1
        best_threshold = float(threshold)

test_labels, test_probs = collect_predictions(model, test_loader)
test_auc = roc_auc_score(test_labels, test_probs)
test_ap = average_precision_score(test_labels, test_probs)
test_preds = (test_probs >= best_threshold).astype(int)
tn, fp, fn, tp = confusion_matrix(test_labels, test_preds, labels=[0, 1]).ravel()
test_sensitivity = tp / max(tp + fn, 1)
test_specificity = tn / max(tn + fp, 1)
```